Master's Thesis

Enhancing Stability in Direct Training of Spiking Neural Networks through Membrane Potential Initialization and Threshold-Robust Surrogate Gradients

Hyunho Kook (국 현 호)

Department of Computer Science and Engineering

Pohang University of Science and Technology

2026

스파이킹 신경망의 직접 학습 안정화를 위한 막전위 초기화와 임계값 강건성 기반 서러게이트 그래디언트

Enhancing Stability in Direct Training of Spiking Neural Networks through Membrane Potential Initialization and Threshold-Robust Surrogate Gradients

Enhancing Stability in Direct Training of Spiking Neural Networks through Membrane Potential Initialization and Threshold-Robust Surrogate Gradients

by

Hyunho Kook
Department of Computer Science and Engineering
Pohang University of Science and Technology

A thesis submitted to the faculty of the Pohang University of Science and Technology in partial fulfillment of the requirements for the degree of Master of Science in the Computer Science and Engineering

Pohang, Korea
01. 15. 2026
Approved by
Eunhyeok Park (Signature)
Academic advisor

Enhancing Stability in Direct Training of Spiking Neural Networks through Membrane Potential Initialization and Threshold-Robust Surrogate Gradients

Hyunho Kook

The undersigned have examined this thesis and hereby certify that it is worthy of acceptance for a master's degree from POSTECH

01. 15. 2026

Committee Chair	Eunhyeok Park	(Seal)
Member	Jungseul Ok	(Seal)
Member	Jae-Joon Kim	(Seal)

MCSE 국 현 호 Hyunho Kook
20242015
Enhancing Stability in Direct Training of Spiking Neural Networks through Membrane Potential Initialization and Threshold-Robust Surrogate Gradients.
스파이킹 신경망의 직접 학습 안정화를 위한 막전위 초기화와 임계값 강건성 기반 서러게이트 그래디언트.
Department of Computer Science and Engineering , 2026, 50p
Advisor : Eunhyeok Park.
Text in English.

ABSTRACT

Recent advancements in the direct training of Spiking Neural Networks (SNNs) have produced high-quality outputs even at early timesteps, paving the way for energy-efficient AI. However, the non-linearity and temporal dependencies of SNNs introduce persistent challenges, notably temporal covariate shift (TCS) and unstable gradient flow when neuron thresholds are learnable.

This thesis enhances the stability of direct SNN training through two innovations: **MP-Init (Membrane Potential Initialization)** and **TrSG (Threshold-Robust Surrogate Gradient)**. MP-Init addresses TCS by aligning the initial membrane potential with its stationary distribution. TrSG stabilizes gradient flow with respect to the threshold voltage during training. We provide a rigorous mathematical analysis of both methods, prove geometric convergence of the membrane potential to a stationary distribution, and characterize the threshold-induced gradient pathologies of existing surrogate-gradient families.

Extensive experiments validate our approach, achieving state-of-the-art accuracy on both static (CIFAR-10/100, ImageNet) and dynamic (DVS-CIFAR10) benchmarks while incurring negligible parameter and computational overhead. The methods further generalize to Transformer-based SNNs and event-based object detection, indicating broad applicability beyond classification.

Contents

List of Figures

List of Tables

I. Introduction

1.1 Motivation

Modern deep neural networks (DNNs) have achieved remarkable accuracy across vision, language, and multimodal tasks, but their energy footprint has grown substantially as model scale has increased. Training and serving large foundation models routinely consumes megawatt-scale power, and even moderately sized inference workloads on edge devices remain difficult to deploy under tight energy budgets. This trajectory motivates a search for fundamentally more efficient computation models.

Spiking Neural Networks (SNNs) offer one such alternative. By transmitting information through discrete spikes over time, SNNs enable efficient, event-driven processing [1, 2, 3, 4, 5, 6], especially when paired with neuromorphic hardware [7, 8, 9]. Recent advances in direct training [10, 11] have significantly improved SNN accuracy with only a handful of timesteps, narrowing the historical efficiency-vs.-accuracy gap. Yet, SNNs still lag behind conventional deep neural networks, and their complex temporal dynamics often hinder performance on large-scale datasets [12, 13]. Further innovations are therefore crucial to ensure reliability and accuracy for real-world adoption.

1.2 Challenges in Direct Training of SNNs

This thesis identifies and addresses two challenges that, despite extensive prior work, remain insufficiently understood.

Challenge 1: Temporal Covariate Shift (TCS). The first challenge is **Temporal Covariate Shift (TCS)** [14], in which internal covariate shifts in activations accumulate over time. Existing remedies such as TEBN [15] and TAB [16] introduce timestep-specific normalization parameters, but they disrupt parameter homogeneity across time and increase both model complexity and computational cost. More importantly, our analysis reveals that these methods fail to mitigate TCS in the membrane potential of leaky-integrate-and-fire (LIF) neurons. Since the membrane potential directly determines neuronal output, neglecting TCS in this component leads to significant accuracy degradation.

Challenge 2: Gradient pathology of learnable thresholds. The second challenge concerns **training the internal parameters** [17, 18, 6] of LIF neurons—chiefly the threshold voltage V_{thr} and the leakage term τ. Despite their crucial role in LIF neuron behavior, these parameters are typically fixed or updated

under strict constraints. We show that existing surrogate-gradient flow heavily depends on the scale of V_{thr}, and that unconstrained updates during training lead to either gradient flooding/starvation or gradient explosion/vanishing. To our knowledge, this issue has not been explicitly recognized prior to this work.

1.3 Contributions

This thesis makes the following contributions:

- **MP-Init (Membrane Potential Initialization).** We trace TCS to the mismatch between the initial membrane potential and its stationary distribution. We prove a discrete-time convergence theorem (Markov-chain Doeblin minorization) showing that the layer-wise membrane potential converges geometrically to a unique stationary distribution under mild assumptions. Building on this, we propose a per-layer running-mean initialization that aligns the initial membrane potential with the stationary mean, eliminating TCS while preserving the standard inference pipeline of LIF SNNs.
- **TrSG (Threshold-Robust Surrogate Gradient).** We classify existing surrogate gradients into Absolute-Scale (AS-SG) and Relative-Scale (RS-SG) families, and analyze how each becomes unstable when V_{thr} is trainable. We then propose TrSG, which keeps the relative-scale active window while canceling the destabilizing $1/V_{\text{thr}}$ factor through a threshold-multiplied forward path. The result is a gradient whose magnitude is invariant to the threshold scale and whose active region adapts naturally with V_{thr}.
- **Comprehensive empirical validation.** On CIFAR-10/100, ImageNet, and DVS-CIFAR10, our combined method achieves state-of-the-art accuracy at competitive timesteps. We further demonstrate that MP-Init and TrSG transfer cleanly to Transformer-based SNN backbones and to event-based object detection, indicating broad applicability.

1.4 Thesis Organization

The remainder of this thesis is organized as follows. Chapter II reviews the background and related work in spiking neural networks, including direct-training methodologies, prior approaches to TCS, and existing strategies for learning internal LIF parameters. Chapter III fixes notation and formalizes the LIF neuron model, the surrogate gradient framework, and the active-neuron convention used throughout the thesis. Chapter IV introduces **MP-Init**: it identifies the root cause of TCS in the membrane potential, presents a rigorous convergence theorem, and develops the running-mean initialization algorithm. Chapter V introduces **TrSG**: it analyzes the threshold sensitivity of existing surrogate gradients, derives the

threshold-robust formulation, and provides a detailed gradient analysis with respect to V_{thr} and τ. Chapter VI reports experimental results on static and event-based datasets, ablation studies, comparisons with state-of-the-art methods, and extensions to advanced architectures and downstream tasks. Chapter VII concludes with a discussion of limitations and directions for future work.

II. Background and Related Work

This chapter situates the contributions of the thesis within the broader literature. We first review the biological motivation and hardware platforms that have made spiking neural networks (SNNs) attractive (Section 2.1). We then survey direct-training methodologies (Section 2.2), existing approaches to temporal covariate shift (Section 2.3), and prior work on learning the internal parameters of LIF neurons (Section 2.4).

2.1 Spiking Neural Networks

2.1.1 Biological Motivation

Biological neurons communicate through asynchronous, all-or-none electrical events called *spikes*. The leaky integrate-and-fire (LIF) abstraction captures the essential dynamics: a neuron integrates incoming current onto its membrane potential, leaks toward a resting value, and emits a spike whenever the membrane potential crosses a threshold. Notably, recent neuroscience evidence [19] indicates that resting membrane potentials differ across distinct cortical regions: neurons in the prefrontal cortex (PFC) exhibit approximately 10 mV higher resting potentials than those in the posterior parietal cortex (PPC), driven by region-specific recurrent synaptic strengths mediated by NMDA receptors. This regional variability in resting potentials motivates the per-layer initialization strategy adopted in this thesis (Chapter IV).

2.1.2 Neuromorphic Hardware

Several specialized accelerators exploit the event-driven nature of spiking computation: TrueNorth [7, 8], Loihi [9], and emerging digital/mixed-signal designs all process spikes asynchronously and skip computation for inactive neurons, yielding orders-of-magnitude lower energy than dense matrix-multiplication accelerators. A practical consequence is that SNN training methods that introduce extra timestep-dependent state, or that depart from the canonical LIF inference pipeline, can be difficult to deploy. The methods in this thesis preserve the standard LIF inference pipeline so that hardware compatibility is unaffected (cf. Section 5.4.1).

2.2 Direct Training of SNNs

Surrogate gradient (SG) descent enables end-to-end training of SNNs by approximating the non-differentiable firing function [20, 10, 11]. Some studies focus on the impact of hyperparameters and the

shape of surrogate derivatives [21, 22], while more recent advances introduce differentiable surrogate gradient functions [22] and Temporal Efficient Training (TET) losses [23]. Other approaches regulate the membrane potential through additional loss terms [24, 25] or through batch-normalization variants [26]. Methods that modify LIF dynamics themselves [27, 28, 29] have also been explored, although they may be incompatible with existing neuromorphic hardware [30, 12].

2.3 Temporal Covariate Shift

The TCS problem was first identified in BNTT [14] and later addressed by methods such as TEBN [15] and TAB [16], which adapt normalization layers on a per-timestep basis. While these methods alleviate TCS in activations, they increase model complexity with additional parameters and, more critically, overlook the persistent shift in membrane potential, a primary source of TCS during inference (see Section 4.3).

IMP+LTS [31] proposed training the initial membrane potential for every neuron to enhance representational power. However, it does not directly address TCS and incurs a large parameter overhead scaling with the number of neurons $\mathcal{O}(N)$. By contrast, our approach initializes membrane potentials at the layer level $\mathcal{O}(L)$ ($L \ll N$), directly targeting TCS while remaining lightweight.

More recently, MPS [32] reinterpreted SNNs from an ensemble-learning perspective, arguing that excessive differences in membrane potential distributions across timesteps destabilize subnetworks. Their smoothing and guidance strategies improve temporal consistency and yield accuracy gains; however, these methods modify the LIF update rule itself and require additional smoothing coefficients and guidance losses. Both training and inference therefore diverge from the standard LIF neuron model. In contrast, our approach preserves the original inference pipeline: MP-Init adds negligible overhead during training while inference remains compatible with neuromorphic hardware [9].

2.4 Learning Internal Parameters of LIF Neurons

Recent studies have explored training internal parameters such as the threshold voltage and the leakage term via gradient descent [17, 18, 6]. PLIF [17] optimizes the leakage term, LTMD [18] adjusts the threshold voltage, and DIET-SNN [6] learns both. However, these works provide limited analysis of the gradient flow associated with learnable parameters, leaving questions about training stability unanswered. In contrast, we find that when the threshold voltage becomes trainable, surrogate-gradient methods can exhibit instability—producing misaligned, exploding, or vanishing gradients (Section 5.3). To overcome these issues we propose, in Chapter V, a novel surrogate-gradient formulation that remains stable when learning the threshold voltage and ensures robust convergence.

2.5 Summary

Existing methods either (i) address TCS only at the activation level while ignoring its origin in the membrane potential, or (ii) train the threshold voltage without a principled treatment of the resulting gradient pathology. The remainder of this thesis closes both gaps.

III. Preliminaries

This chapter fixes the notation used throughout the thesis. We introduce the discrete-time leaky integrate-and-fire (LIF) neuron model (Section 3.1) and clarify the convention by which silent neurons are excluded from our statistical analyses (Section 3.2).

3.1 The LIF Neuron Model

The LIF model [33] is a widely used spiking neural model that captures key behaviors of biological neurons, including spike firing and membrane-potential decay. In discrete time $t \in \mathbb{N}$, the LIF neuron's behavior is described by the following three equations.

Membrane integration.

$$M^l[t] \;=\; \left(1 - \frac{1}{\tau^l}\right) U^l[t-1] + \frac{1}{\tau^l} I^l_{\text{in}}[t], \tag{3.1}$$

Spike generation.

$$S^l[t] \;=\; \mathcal{H}\Big(M^l[t] - V^l_{\text{thr}}\Big), \tag{3.2}$$

Reset.

$$U^l[t] \;=\; \begin{cases} M^l[t] - V^l_{\text{thr}} \cdot S^l[t], & \text{(soft reset)} \\ M^l[t] \cdot (1 - S^l[t]). & \text{(hard reset)} \end{cases} \tag{3.3}$$

Here l is the layer index, $\mathcal{H}(\cdot)$ is the Heaviside step function, $M^l[t]$ is the post-integration membrane potential, $U^l[t]$ is the post-reset membrane potential, $S^l[t] \in \{0, 1\}$ is the spike output, V^l_{thr} is the threshold voltage, and τ^l is the leakage time constant. The presynaptic input current is

$$I^l_{\text{in}}[t] \;=\; W^l_{l-1}\, O^{l-1}[t],$$

where W^l_{l-1} is the synaptic weight matrix from layer $l-1$ to l, and $O^{l-1}[t]$ is the output of the previous layer. In direct training, this output typically equals the binary spike itself, $O^l[t] = S^l[t]$ [10]. Following standard practice [33], the leakage constant τ^l can be absorbed into the synaptic weights via the rescaling $W^l_{l-1} := \frac{1}{\tau^l} W^l_{l-1}$ in Eq. (3.1).

Soft vs. hard reset. The choice between soft and hard reset in Eq. (3.3) affects post-firing dynamics. Under a soft reset [34], the membrane potential decreases by V^l_{thr} when a spike fires; under a hard reset [10], the membrane potential resets to zero. Both regimes are widely used in the literature, and the methods proposed in this thesis are effective in either case.

3.2 Active Neurons and Silent Neurons

In our analysis of TCS we focus only on *active neurons*, i.e., those that fire at least once during each simulation. Prior work [35] shows that a substantial fraction of neurons in trained SNNs remain subthreshold throughout a simulation; we refer to these as *silent* neurons. Silent neurons are irrelevant to our statistical analysis: they neither contribute to information propagation nor exhibit statistical properties similar to those of active neurons. Therefore, when computing membrane-potential statistics in this thesis, we exclude silent neurons.

3.3 Notation Summary

Table 3.1 summarizes the symbols used throughout the thesis.

Symbol	**Meaning**
l	Layer index
t	Discrete timestep
$M^l[t]$	Post-integration membrane potential at layer l, time t
$U^l[t]$	Post-reset membrane potential
$S^l[t] \in \{0, 1\}$	Spike output
$O^l[t]$	Layer output (typically $= S^l[t]$ in direct training)
V_{thr}^l	Threshold voltage at layer l
τ^l	Leakage time constant at layer l
$I_{\text{in}}^l[t]$	Presynaptic input current
W_{l-1}^l	Synaptic weight matrix from layer $l-1$ to l
$\mathcal{H}(\cdot)$	Heaviside step function
π	Stationary distribution of $U[t]$ (Markov chain)
T	Total number of timesteps in the simulation window

Table 3.1: Notation used throughout this thesis.

IV. Membrane Potential Initialization (MP-Init)

This chapter presents **MP-Init**, the first contribution of this thesis. Section 4.1 motivates the method by examining why existing TCS remedies are insufficient. Sections 4.2 and 4.3 empirically and theoretically pinpoint the root cause of TCS in the *membrane potential*, and Sections 4.4 and 4.5 establish the convergence properties used to derive the algorithm in Section 4.6. Section 4.10 discusses biological plausibility, and Section 4.11 closes the chapter.

4.1 Motivation

TCS [14, 15, 16] limits the performance of SNNs during inference, particularly when batch normalization (BN) is employed. Despite BN's capability to normalize activations during training, the fixed normalization statistics used at inference cause activation drift across timesteps and accuracy degradation. Previous methods [15, 16] have tried to address this issue through timestep-dependent normalization, but these approaches remain superficial: they fail to resolve the underlying cause—drift in the membrane potential of LIF neurons.

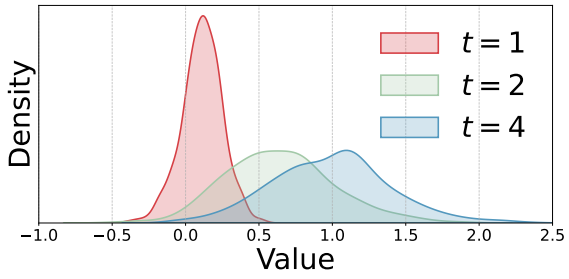
To tackle this issue, we propose **MP-Init (Membrane Potential Initialization)**, an efficient method that directly targets TCS in the membrane potential. MP-Init ensures stable activation distributions throughout inference by initializing the membrane potential of spiking neurons in each layer to match its stationary distribution. This eliminates the need for timestep-dependent normalization and improves final performance by aligning training and inference statistics.

4.2 Empirical Evidence: TCS in the Membrane Potential

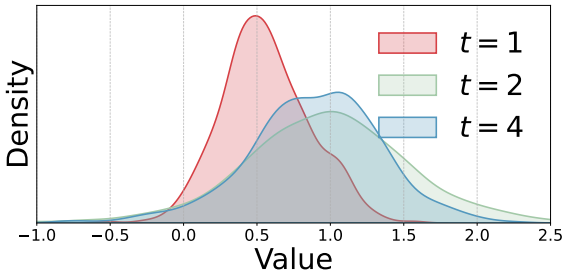
4.3 Root Cause of TCS

We claim that TCS in activations fundamentally originates from TCS in the membrane potential. By Eq. (3.2), the spike output depends directly on the membrane potential; if the membrane potential drifts significantly over timesteps, the activation distribution shifts correspondingly.

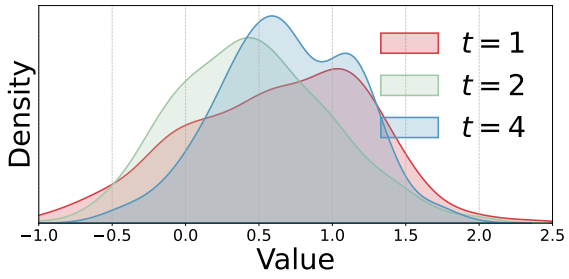
Figure 4.1 visualizes the membrane potential at each timestep and validates this claim. tdBN, which does not account for TCS, exhibits a clear temporal drift, leading to TCS in the activation distribution (Figure 4.2). While TEBN and TAB attempt to stabilize activations through timestep-dependent normalization, the membrane potential itself continues to drift, preserving TCS in activations.



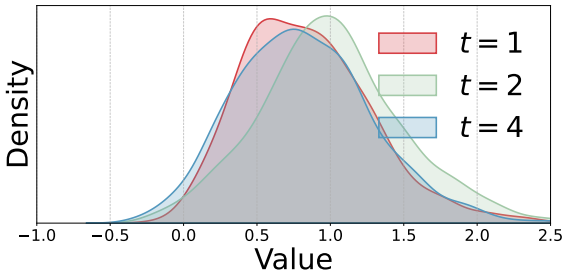
(a) tdBN



(b) TEBN

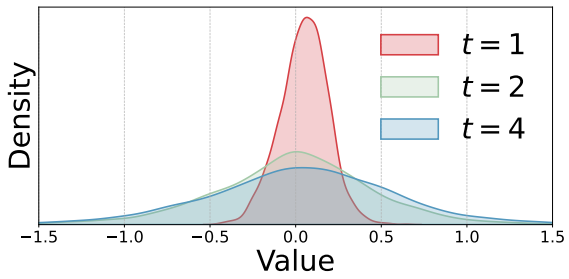


(c) TAB

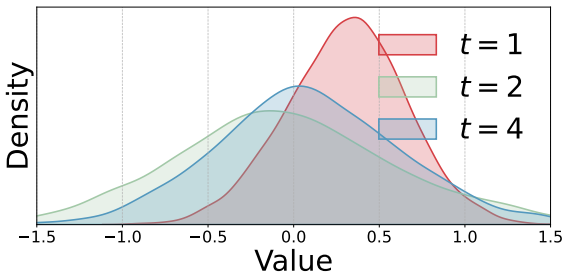


(d) MP-Init (ours)

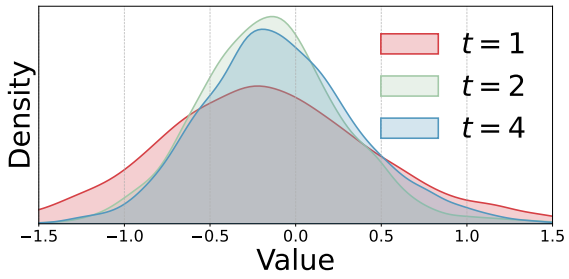
Figure 4.1: Membrane potential distributions across timesteps at the third spiking layer of ResNet-19 on CIFAR-100. Existing TCS-aware normalizations (TEBN, TAB) still leave a temporal drift in the membrane potential, while MP-Init aligns the per-timestep distributions.



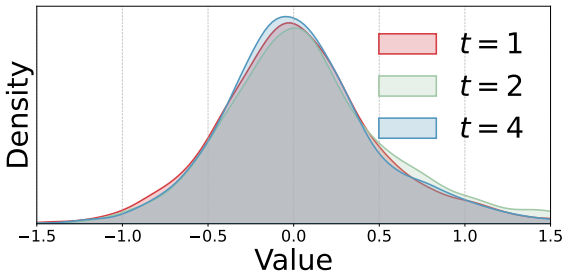
(a) tdBN



(b) TEBN



(c) TAB



(d) MP-Init (ours)

Figure 4.2: Activation (presynaptic input) distributions of the convolutional layer following the third spiking layer of ResNet-19 on CIFAR-100. The drift visible in TEBN and TAB collapses under MP-Init.

4.4 Convergence of the Membrane Potential

While the empirical evidence above demonstrates the existence of TCS in the membrane potential, a theoretical perspective provides a deeper understanding. In this section we present the core principles explaining *why* the membrane potential drifts.

We prove that the layer-wise distribution of the membrane potential converges to a (possibly non-trivial) stationary distribution. The continuous-time analogue was previously established for stable input statistics [36]; we extend the result to the discrete-time model and show that the same convergence property holds.

Assumption IV.1 (i.i.d. inputs and bounded membrane potential)**.** *Let $\{I_{in}[t]\}_{t\geq 1}$ be a sequence drawn from an i.i.d. distribution. Furthermore, suppose there exist real constants $u^- < u^+$ such that the membrane potential $U[t]$ of active neurons in each layer remains within $[u^-, u^+]$ for all t.*

The i.i.d. assumption is reasonable: inputs $\{I_{\text{in}}[t]\}_{t\geq 1}$ are drawn repeatedly from a dataset or from a normalized layer. While soft-reset LIF dynamics do not strictly prevent unbounded growth, well-trained SNNs rarely exhibit it in active neurons. An empirical validation of assumption IV.1 is provided in Section 4.7.

Theorem IV.1 (Geometric convergence to a stationary distribution)**.** *Under assumption IV.1, the sequence $\{U[t]\}_{t\geq 0}$ forms a Markov chain with transition kernel P. This chain satisfies Doeblin's minorization condition, and consequently admits a unique stationary distribution π. Moreover, the total-variation (TV) distance between $U[t]$ and π decreases exponentially fast in t.*

Proof. The proof proceeds in four steps: (Step 1) verify the Markov property; (Step 2) characterize the transition kernel; (Step 3) establish Doeblin's minorization condition [37]; and (Step 4) derive exponential decay in TV distance.

Step 1: Markov property. The dynamics of $U[t]$ are

$$U[t+1] \;=\; f\big(U[t],\, I_{\text{in}}[t+1]\big), \tag{4.1}$$

where $f(\cdot)$ is a deterministic function of the current membrane potential $U[t]$ and the new input $I_{\text{in}}[t+1]$. Since $\{I_{\text{in}}[t]\}_{t\geq 1}$ is i.i.d., the future state $U[t+1]$ depends solely on the current state $U[t]$ and not on earlier states:

$$\Pr\big(U[t+1] \in A \,\big|\, U[t], \ldots, U[0]\big) \;=\; \Pr\big(U[t+1] \in A \,\big|\, U[t]\big). \tag{4.2}$$

Hence $\{U[t]\}_{t\geq 0}$ is a Markov chain.

Step 2: Transition kernel. The transition kernel $P(x, A)$ is the probability that $U[t+1]$ falls in the measurable set A given $U[t] = x$:

$$P(x, A) \;=\; \Pr\big(U[t+1] \in A \,\big|\, U[t] = x\big) \;=\; \int_A p(x, y)\, \mathrm{d}y, \tag{4.3}$$

where the transition density $p(x, y)$ encapsulates the dynamics induced by $f(\cdot)$ and the i.i.d. input distribution.

Step 3: Doeblin's minorization condition. We show that there exist a constant $\epsilon > 0$ and a probability measure $\mu(\cdot)$ on the state space $S = [u^-, u^+]$ such that

$$P(x, A) \;\geq\; \epsilon\, \mu(A) \qquad \forall\, x \in S,\ \forall\, A \subseteq S \text{ measurable.} \tag{4.4}$$

Because S is bounded by assumption IV.1 and the transition density $p(x, y)$ is continuous (the i.i.d. inputs ensure a smooth density on S), there exists a positive lower bound $\delta > 0$ such that $p(x, y) \geq \delta$ for all $(x, y) \in S \times S$. Hence

$$P(x, A) \;=\; \int_A p(x, y)\, \mathrm{d}y \;\geq\; \delta \cdot \lambda(A), \tag{4.5}$$

where λ is the Lebesgue measure on S. Letting $\mu(A) = \lambda(A)/\lambda(S)$ be the uniform probability measure on S and $\epsilon = \delta\, \lambda(S)$ yields Eq. (4.4).

Step 4: Exponential decay of TV distance. A direct consequence of Doeblin's minorization condition (see, e.g., [37]) is that the n-step transition $P^n(x, \cdot)$ converges geometrically to a unique stationary distribution π:

$$\left\| P^n(x, \cdot) - \pi(\cdot) \right\|_{\mathrm{TV}} \;\leq\; C\,(1 - \epsilon)^n \tag{4.6}$$

for some constant $C > 0$ depending on the initial state. Since $1 - \epsilon < 1$, the TV distance decreases exponentially in n, simultaneously establishing the existence and uniqueness of the stationary distribution. □

Theorem IV.1 indicates that TCS in the membrane potential is *inevitable* whenever the initial membrane potential differs from its stationary distribution. Traditionally the membrane potential is initialized to zero, $U[0] = 0$; this zero initialization is a primary contributor to TCS. To mitigate TCS, we therefore must reduce the discrepancy between the initial state and the stationary distribution.

4.5 Optimal Constant Initialization

Theorem IV.1 implies that eliminating the mismatch between the initial state and the stationary distribution is crucial. Directly computing π is intractable; we therefore propose a more practical approach—initialize the membrane potential with a *per-layer constant* that approximates the mean of π.

Lemma IV.2 (Mean optimality among constants)**.** *Let $U^* \sim \pi$ be a sample from the stationary distribution. Among all constants $c \in \mathbb{R}$, the choice $c = \mathbb{E}[U^*]$ minimizes the expected square difference $\mathbb{E}[(U^* - c)^2]$.*

Proof. Expanding the square,

$$\mathbb{E}[(U^* - c)^2] \;=\; \mathbb{E}[U^{*2}] - 2\,\mathbb{E}[U^*]\, c + c^2. \tag{4.7}$$

Differentiating with respect to c and setting the derivative to zero,

$$\frac{\partial \mathbb{E}[(U^* - c)^2]}{\partial c} \;=\; -2\,\mathbb{E}[U^*] + 2c \;=\; 0 \quad\Longrightarrow\quad c \;=\; \mathbb{E}[U^*]. \tag{4.8}$$

The second derivative is $2 > 0$, confirming that the critical point is a minimum. □

Lemma IV.2 suggests that initializing the membrane potential to the stationary mean is optimal for eliminating drift while minimizing variance. The remaining challenge is to approximate $\mathbb{E}[\pi]$ in practice.

A simple solution leverages theorem IV.1: a membrane-potential distribution converges exponentially to π, so the membrane potential at the final timestep $U[T]$ is the closest sample to π. We therefore approximate the per-layer statistic of $\mathbb{E}[\pi]$ using $\mathbb{E}[U[T]]$.

4.6 The MP-Init Algorithm

To implement the idea efficiently, we compute the average membrane potential $U[T]$ at the final timestep for each layer during training. This average is used to update a running mean r, initialized to zero at the beginning of training. Before feeding each batch into the network—both during training and at inference—we initialize the membrane potential of every layer to this running mean, $U[0] = r$. The procedure, called **MP-Init**, requires negligible extra memory or computation. Algorithm 1 summarizes the steps.

Algorithm 1 MP-Init: per-layer running-mean initialization

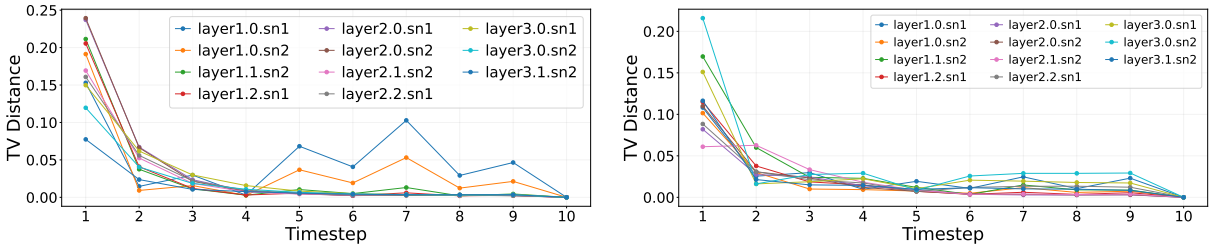
Initialize per-layer running mean $\mu^l \leftarrow 0$ for each layer l
Training phase:
for each batch with simulation window of length T **do**
    Set initial membrane potential $U^l[0] \leftarrow \mu^l$
    Initialize activity mask $\texttt{mask}^l \leftarrow 0$
    for $t = 1$ to T **do**
        Update $M^l[t]$ via Eq. (3.1)
        Compute spikes $S^l[t]$ via Eq. (3.2)
        Reset $U^l[t]$ via Eq. (3.3)
        $\texttt{mask}^l \leftarrow \texttt{mask}^l$ OR $(S^l[t] > 0)$
    end for
    $\bar{U}^l \leftarrow \mathrm{mean}\{U^l[T] : \texttt{mask}^l = 1\}$ ▷ active neurons only
    $\mu^l \leftarrow \beta\,\mu^l + (1-\beta)\,\bar{U}^l$ ▷ EMA update, $\beta = 0.9$
end for
Inference phase: keep μ^l fixed; initialize $U^l[0] \leftarrow \mu^l$ at every batch.

As illustrated in Figures 4.1 and 4.2, MP-Init effectively resolves both TCS in the membrane potential and TCS in activations, yielding consistent distributions across timesteps. Importantly, MP-Init is straightforward to implement, does not modify BN layers, and does not introduce complex training pipelines—unlike TEBN [15] or TAB [16]. This simplicity significantly enhances usability across applications without altering existing recipes.

4.7 Empirical Validation of Assumption IV.1

We provide three checks of assumption IV.1 in our trained networks: (i) the inputs are nearly identically distributed across timesteps, (ii) they are approximately independent, and (iii) the membrane potential of active neurons remains bounded.

(i) Identical distribution. We compute the TV distance between the presynaptic-input distributions at timesteps $t = 1, \ldots, 9$ and the reference distribution at $t = 10$. As shown in Figure 4.3, distances are consistently small (< 0.1 after $t = 2$) on both CIFAR-100 and DVS-CIFAR10, indicating that the input distribution stabilizes quickly and remains nearly identical across timesteps.

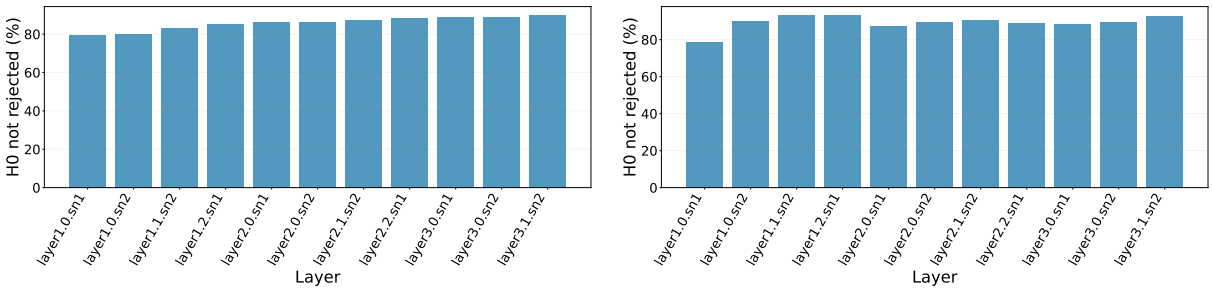


(a) CIFAR-100 (b) DVS-CIFAR10

Figure 4.3: TV distance between the presynaptic-input distribution at each timestep and the reference at $t = 10$.

(ii) Independence of inputs. We perform a Ljung–Box test at lag 1 on the per-timestep presynaptic-input statistics. The test fails to reject the null hypothesis of zero serial correlation in 80–90% of neurons on both CIFAR-100 (Figure 4.4a) and DVS-CIFAR10 (Figure 4.4b), supporting the i.i.d. assumption.

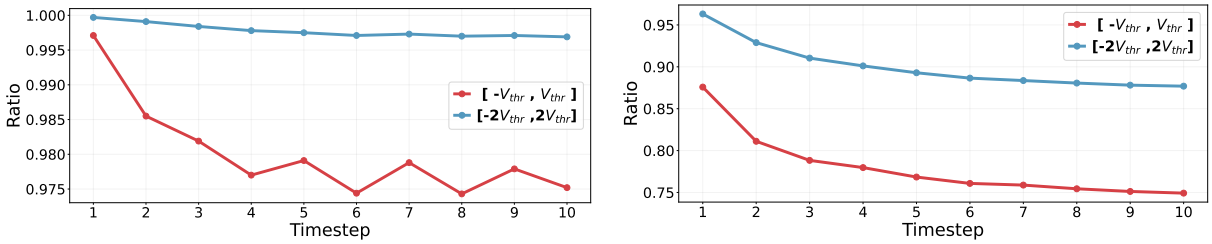
(iii) Bounded membrane potential. We measure the empirical fraction of active neurons whose $|U^l[t]|$ stays within a per-layer bound. As shown in Figure 4.5, over 97% of membrane potentials remain within $\pm V_{\text{thr}}$ at all timesteps, and nearly all values lie within $\pm 2V_{\text{thr}}$, confirming that the dynamics are effectively bounded in practice.



(a) CIFAR-100, ResNet-19

(b) DVS-CIFAR10, ResNet-19

Figure 4.4: Ljung–Box test (lag 1) for the presynaptic input across layers and timesteps. Bars show the percentage of neurons for which independence cannot be rejected.



(a) CIFAR-100

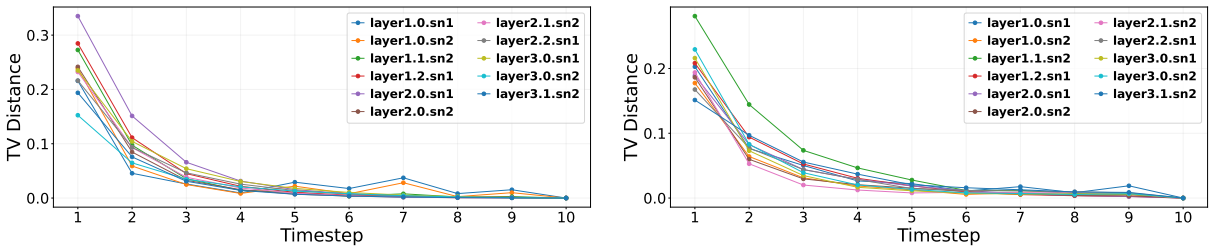
(b) DVS-CIFAR10

Figure 4.5: Empirical boundedness ratio of $|U^l[t]|$ for active neurons.

Together these three diagnostics confirm that assumption IV.1 holds empirically in our trained networks, validating the theoretical conditions of theorem IV.1.

4.8 Convergence Speed of the Membrane Potential

Theorem IV.1 predicts an exponential convergence of $U[t]$ toward the stationary distribution π. We empirically verify this by computing the TV distance between the membrane-potential distribution at each timestep and the reference distribution at $t = 10$. As shown in Figure 4.6, deviations are large at $t = 1$ but shrink rapidly, already becoming negligible (< 0.1) by $t = 3$ on both CIFAR-100 and DVS-CIFAR10. This rapid convergence is highly desirable for SNNs, since reducing the number of timesteps directly improves both energy efficiency and latency. The benefit is especially pronounced at very low timesteps: at $T = 2$, MP-Init delivers a **1.66 %pt** accuracy improvement over the baseline (Table 4.1), enabling reliable inference even under aggressive latency constraints.



(a) CIFAR-100 (b) DVS-CIFAR10

Figure 4.6: Total-variation distance between $U[t]$ and the reference at $t = 10$ as a function of t. The decay is clearly geometric, in agreement with the conclusion of theorem IV.1.

Method	**w/o MP-Init (%)**	**w/ MP-Init (%)**
tdBN ($T = 2$)	72.35 ± 0.18	$\mathbf{74.01 \pm 0.33}$

Table 4.1: Effect of MP-Init at early timesteps on CIFAR-100 using ResNet-19.

4.9 Convergence Rate as a Function of τ and V_{thr}

Beyond establishing convergence, theorem IV.1 also offers insight into the convergence *rate*, governed by Doeblin's constant ϵ in Eq. (4.6). We now analyze how the LIF parameters τ and V_{thr} influence this rate.

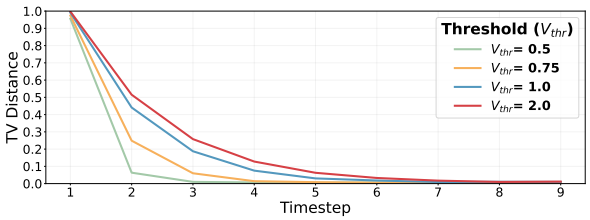
Consider the uniform measure μ representing a uniform distribution over $[u^-, u^+]$. For simplicity, take $U[t-1] = 0$, so that $U[t] = \frac{1}{\tau} I_{\text{in}}[t]$. Under this assumption, Eq. (4.4) can be expressed as

$$\Pr\Big(U[t] = \tfrac{1}{\tau} I_{\text{in}}[t] \in A\Big) \;\geq\; \epsilon\, \mu(A). \tag{4.9}$$

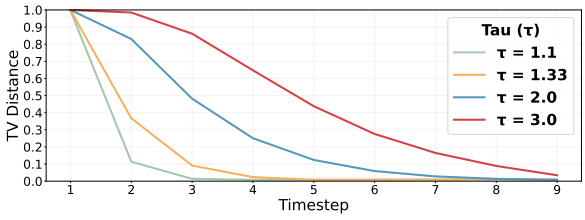
Decreasing τ enlarges the maximum permissible ϵ, which by Eq. (4.6) accelerates convergence. Similarly, reducing V_{thr} causes the membrane potential to cross threshold and reset more frequently; this narrows the effective range $[u^-, u^+]$, again increasing ϵ and accelerating convergence.

We validate this analysis with soft-reset LIF neurons driven by Gaussian inputs, varying V_{thr} and τ and computing the TV distance between the distribution at each timestep and the final distribution (Figure 4.7). The results confirm that smaller V_{thr} and τ alleviate TCS. However, setting these parameters to extremely small values is not universally beneficial: an excessively low V_{thr} produces unstable firing rates, while an excessively low τ degrades the neuron's ability to integrate input over time, effectively reducing it to binary quantization. A careful balance is therefore required.

This analysis suggests a natural connection between MP-Init and TrSG (introduced in Chapter V): rather than relying on manual tuning, training V_{thr} and τ as learnable parameters lets the network discover the operating point itself. As shown later in Section 6.15, TrSG drives V_{thr} and τ toward moderately low values that combine fast convergence with stable dynamics.



(a) Impact of varying V_{thr} on convergence speed.



(b) Impact of varying τ on convergence speed.

Figure 4.7: TV distance of the membrane-potential distribution between timestep 10 and previous timesteps under different V_{thr} (left) and τ (right). Inputs are sampled from a Gaussian distribution.

4.10 Biological Plausibility

A neuroscience study [19] reports that resting membrane potentials (RMPs) differ across distinct cortical regions: neurons in the prefrontal cortex (PFC) exhibit ~ 10 mV higher RMPs than neurons in the posterior parietal cortex (PPC), driven by region-specific recurrent synaptic strengths mediated by NMDA receptors. These regional differences imply that the stationary membrane-potential distribution may inherently vary across neural populations and brain areas. The layer-wise initialization of MP-Init mirrors this intrinsic regional variability, lending biological plausibility to the proposed scheme.

4.11 Summary

This chapter identified the membrane potential as the root cause of TCS in SNNs (Sections 4.2 and 4.3), proved that the membrane potential converges geometrically to a stationary distribution (Section 4.4), and used this fact to derive a per-layer running-mean initialization, MP-Init, that eliminates TCS at negligible cost (Section 4.6). We further showed that MP-Init accelerates inference at low timesteps (Section 4.8) and explained how the LIF parameters τ and V_{thr} modulate the convergence rate (Section 4.9). The analysis of how MP-Init affects gradient flow during training is deferred to Section 6.7 alongside the parallel analysis for TrSG, where the two methods can be compared directly.

V. Threshold-Robust Surrogate Gradient (TrSG)

This chapter presents the second contribution of the thesis: **TrSG**, a surrogate-gradient (SG) formulation whose gradient magnitude is invariant to the threshold scale and whose active region adapts naturally with V_{thr}. Section 5.2 reviews surrogate-gradient descent. Section 5.3 dissects two existing families—Absolute-Scale (AS-SG) and Relative-Scale (RS-SG)—and shows how each becomes unstable when V_{thr} is trainable. Section 5.4 derives TrSG, and Sections 5.5 and 5.6 provide a detailed gradient analysis with respect to V_{thr} and τ together with compatibility checks across surrogate shapes.

5.1 Motivation

Surrogate-gradient (SG) descent [10] enables gradient-based optimization of SNNs by replacing the non-differentiable spiking function with a differentiable surrogate near the threshold. This permits effective gradient descent and often improves performance at early timesteps. However, our analysis shows that SG used for internal neuronal parameters [17, 18, 6]—particularly V_{thr}—makes training highly sensitive to the *scale* of V_{thr}. While careful tuning can yield strong accuracy, this dependency exposes a fundamental limitation of current SG-based training.

In this chapter, we identify—*to our knowledge for the first time*—that prior SG designs exhibit strong sensitivity of gradient flow to V_{thr}. To address it, we introduce **TrSG (Threshold-Robust SG)**, which maintains stable gradients regardless of the threshold value. TrSG is simple to implement, adds negligible overhead, and is compatible with a wide range of surrogate functions.

5.2 Background: Surrogate Gradient Descent

By Eq. (3.2), an LIF neuron emits a spike when the membrane potential crosses the threshold. Let $H(x)$ denote the Heaviside step. The spike can be written as

$$S^l[t] \;=\; H\big(x(M^l[t], V^l_{\text{thr}})\big),$$

where the *argument* $x(\cdot)$ encodes how the membrane potential $M^l[t]$ and the threshold V^l_{thr} are combined (e.g., $x = M^l[t] - V^l_{\text{thr}}$). Because $H(\cdot)$ is non-differentiable and $H'(\cdot)$ is a Dirac delta, SG training replaces H with a smooth surrogate f whose derivative locally approximates H':

$$H'(x) \;\approx\; f'(x) \quad \text{near } x = 0.$$

Applying the chain rule yields the working gradient

$$\frac{\partial S^l[t]}{\partial M^l[t]} \;=\; H'\big(x\big)\,\frac{\partial x}{\partial M^l[t]} \;\approx\; f'\big(x\big)\,\frac{\partial x}{\partial M^l[t]}. \tag{5.1}$$

The *choice* of $x(\cdot)$ is crucial—especially when V_{thr}^l is learnable—because it determines both the active region of nonzero gradients and the scaling of their magnitudes.

Reviewing the literature, SG-based methods fall into two families based on how they define $x(M^l[t], V_{\text{thr}}^l)$: *Absolute-Scale* (AS-SG) and *Relative-Scale* (RS-SG). The distinction becomes significant once the threshold is trained.

5.3 Two Families of Surrogate Gradients

To make the scaling effects explicit, we analyze a representative surrogate with a rectangular derivative

$$f'(x) \;=\; \frac{1}{\gamma}\,\mathbb{1}\Big(|x| < \tfrac{\gamma}{2}\Big),$$

where $\gamma > 0$ controls the width of the region where gradients flow. The phenomena described below are *not* specific to this choice; any surrogate exhibits the same issues. Figure 5.1 visualizes the membrane-potential distribution under two different threshold scales: small ($V_{\text{thr}}^l \ll 1$) and large ($V_{\text{thr}}^l \gg 1$).

5.3.1 AS-SG: Absolute-Scale Surrogate Gradient

Many works [38, 23, 17, 18] take

$$x \;=\; M^l[t] - V_{\text{thr}}^l, \tag{5.2}$$

which gives

$$f'\big(M^l[t] - V_{\text{thr}}^l\big) \;=\; \frac{1}{\gamma}\,\mathbb{1}\Big(\big|M^l[t] - V_{\text{thr}}^l\big| < \tfrac{\gamma}{2}\Big), \tag{5.3}$$

and thus

$$\frac{\partial S^l[t]}{\partial M^l[t]} \;\approx\; f'\big(M^l[t] - V_{\text{thr}}^l\big). \tag{5.4}$$

Here the active window $\{M^l[t] \in (V_{\text{thr}}^l - \frac{\gamma}{2},\, V_{\text{thr}}^l + \frac{\gamma}{2})\}$ has a *fixed* width γ, with magnitude independent of V_{thr}^l. When V_{thr}^l is small, the window is excessively wide relative to the membrane-potential spread, so gradients keep flowing far from the true firing boundary—a regime we term *gradient flooding*. Conversely, when V_{thr}^l is large, the fixed-width window covers only a tiny fraction of the distribution, and most gradients vanish—*gradient starvation*.

5.3.2 RS-SG: Relative-Scale Surrogate Gradient

Other works normalize by the threshold, either by setting $\gamma = V_{\text{thr}}^l$ [39] or by defining $x = \frac{M^l[t]}{V_{\text{thr}}^l} - 1$ as in [6]. In this case,

$$f'\Big(\tfrac{M^l[t]}{V_{\text{thr}}^l} - 1\Big) \;=\; \frac{1}{\gamma}\,\mathbb{1}\Big(\Big|\tfrac{M^l[t]}{V_{\text{thr}}^l} - 1\Big| < \tfrac{\gamma}{2}\Big), \tag{5.5}$$

so the active window $\{M^l[t] \in ((1-\frac{\gamma}{2})V_{\text{thr}}^l,\, (1+\frac{\gamma}{2})V_{\text{thr}}^l)\}$ scales with V_{thr}^l. However, the chain rule introduces a sensitivity of the gradient magnitude to the threshold:

$$\frac{\partial S^l[t]}{\partial M^l[t]} \;\approx\; \frac{1}{V_{\text{thr}}^l}\, f'\Big(\tfrac{M^l[t]}{V_{\text{thr}}^l} - 1\Big). \tag{5.6}$$

Thus, small thresholds amplify gradients excessively (*gradient explosion*), whereas large thresholds suppress them severely (*gradient vanishing*).

5.4 Threshold-Robust Surrogate Gradient

We propose **TrSG**, a simple modification that preserves the desirable *relative* window while canceling the problematic $1/V_{\text{thr}}^l$ factor in the gradient. During training we (i) keep the relative-scale argument,

$$x \;=\; \frac{M^l[t]}{V_{\text{thr}}^l} - 1,$$

and (ii) multiply the spike by the threshold on the forward path:

$$O^l[t] \;=\; V_{\text{thr}}^l \cdot S^l[t]. \tag{5.7}$$

Then,

$$\frac{\partial O^l[t]}{\partial M^l[t]} \;=\; V_{\text{thr}}^l\, \frac{\partial S^l[t]}{\partial M^l[t]} \;\approx\; V_{\text{thr}}^l \cdot \Big(\frac{1}{V_{\text{thr}}^l} f'(x)\Big) \;=\; f'(x), \tag{5.8}$$

so the gradient magnitude becomes *threshold-invariant* while the active window still adapts with V_{thr}^l.

5.4.1 Inference Remains Binary

The threshold scaling in Eq. (5.7) is training-only. At test time we revert to $S^l[t] \in \{0, 1\}$ and absorb V_{thr}^l into the next layer through a simple weight rescaling $W_l^{l+1} \leftarrow V_{\text{thr}}^l\, W_l^{l+1}$. Inference therefore uses standard binary spikes and a standard LIF neuron, preserving compatibility with existing neuromorphic hardware.

Summary. TrSG unifies a relative-scale argument with threshold multiplication on the forward path, ensuring stable gradients across thresholds. This design eliminates both the flooding/starvation issue of AS-SG and the vanishing/explosion problem of RS-SG.

5.5 Detailed Gradient Analysis

In Sections 5.3 and 5.4 our analysis of threshold-related instabilities centered on $\partial S^l[t]/\partial M^l[t]$, i.e., how the spike output changes with respect to the membrane potential. However, similar issues arise when considering partial derivatives with respect to the learnable internal parameters V_{thr}^l and τ^l. We now derive these gradients explicitly and show that AS-SG and RS-SG fail along these pathways as well, while TrSG remains stable.

5.5.1 Gradient with Respect to V_{thr}^l

Recall from Eq. (3.2) that $S^l[t] = \mathcal{H}\big(M^l[t] - V_{\text{thr}}^l\big)$. When approximating $\mathcal{H}$ by a differentiable surrogate f_{sr}, existing methods define the argument x in either absolute-scale or relative-scale form.

AS-SG. With $x = M^l[t] - V_{\text{thr}}^l$,

$$\frac{\partial S^l[t]}{\partial V_{\text{thr}}^l} \;=\; f'_{sr}(x)\,\Big(\tfrac{\partial M^l[t]}{\partial V_{\text{thr}}^l} - 1\Big). \tag{5.9}$$

This update does not adapt its magnitude to V_{thr}^l. Consequently, if $|V_{\text{thr}}^l|$ becomes very large or very small, the gradient can vanish or dominate excessively.

RS-SG. With $x = \frac{M^l[t]}{V_{\text{thr}}^l} - 1$, the chain rule yields

$$\frac{\partial S^l[t]}{\partial V_{\text{thr}}^l} \;\approx\; f'_{sr}\Big(\tfrac{M^l[t]}{V_{\text{thr}}^l} - 1\Big)\,\frac{\partial}{\partial V_{\text{thr}}^l}\Big(\tfrac{M^l[t]}{V_{\text{thr}}^l} - 1\Big), \tag{5.10}$$

which introduces factors proportional to $\frac{1}{V_{\text{thr}}^l}$. Tiny V_{thr}^l explodes the gradient; huge V_{thr}^l vanishes it.

TrSG. TrSG circumvents both extremes by adopting the relative-scale argument and multiplying V_{thr}^l on the forward path:

$$O^l[t] \;=\; V_{\text{thr}}^l\, S^l[t]. \tag{5.11}$$

The resulting gradient of the loss $\mathcal{L}$ with respect to the threshold combines the direct effect of the multiplication and the indirect effect via the surrogate:

$$\frac{\partial \mathcal{L}}{\partial V_{\text{thr}}^l} \;=\; \frac{\partial \mathcal{L}}{\partial O^l[t]}\Big(S^l[t] \;+\; V_{\text{thr}}^l\,\tfrac{\partial S^l[t]}{\partial V_{\text{thr}}^l}\Big). \tag{5.12}$$

The threshold multiplication cancels the problematic $\frac{1}{V_{\text{thr}}^l}$ factor in the chain rule, keeping the gradient with respect to V_{thr}^l stable across the full range of threshold scales.

5.5.2 Gradient with Respect to τ^l

The leakage constant τ^l enters $M^l[t]$ through the discrete LIF update of Eq. (3.1):

$$M^l[t] \;=\; \Big(1 - \tfrac{1}{\tau^l}\Big) U^l[t-1] + \tfrac{1}{\tau^l} I^l_{\text{in}}[t]. \tag{5.13}$$

Since τ^l appears only in $M^l[t]$, it influences $S^l[t]$ indirectly through the chain

$$\frac{\partial S^l[t]}{\partial \tau^l} \;=\; \frac{\partial S^l[t]}{\partial M^l[t]} \;\times\; \frac{\partial M^l[t]}{\partial \tau^l}. \tag{5.14}$$

If $\partial S^l[t]/\partial M^l[t]$ is poorly scaled (as in standard AS-SG or RS-SG with extreme V^l_{thr}), the corresponding gradient with respect to τ^l can also explode or vanish. By stabilizing the membrane-potential pathway, TrSG simultaneously stabilizes the τ^l pathway, ensuring well-behaved updates of the leakage constant.

Summary. TrSG's threshold-multiplication mechanism, combined with its relative-scale argument, ensures that gradients remain appropriately scaled not only along $M^l[t]$ but also along the V^l_{thr} and τ^l pathways. This provides a uniformly stable foundation for end-to-end training of LIF SNNs with multiple learnable internal parameters.

5.6 Compatibility Across Surrogate Shapes

The TrSG construction is independent of the specific surrogate-derivative shape $f'(\cdot)$. We highlight four common choices and confirm that the threshold-invariance property of Eq. (5.8) carries through.

Rectangular. $f'(x) = \frac{1}{\gamma}\,\mathbb{1}(|x| < \frac{\gamma}{2})$.

Triangular. $f'(x) = \frac{1}{\gamma^2}\,\max(0, \gamma - |x|)$.

Arctan. $f'(x) = \frac{1}{\pi}\,\frac{1}{1+(\pi x)^2}$.

Sigmoid. $f'(x) = \alpha\,\sigma(\alpha x)\,(1 - \sigma(\alpha x))$.

For all four, the chain-rule cancellation in Eq. (5.8) eliminates the $1/V_{\text{thr}}$ factor; Chapter VI verifies that empirical training behavior aligns with this prediction.

5.7 Summary

This chapter classified existing SG methods into AS-SG and RS-SG, exposed the threshold-induced gradient pathologies of each, and introduced TrSG, which restores threshold invariance with a single forward-path modification. The technique is shape-agnostic, training-only, and inference-compatible with standard LIF neurons.

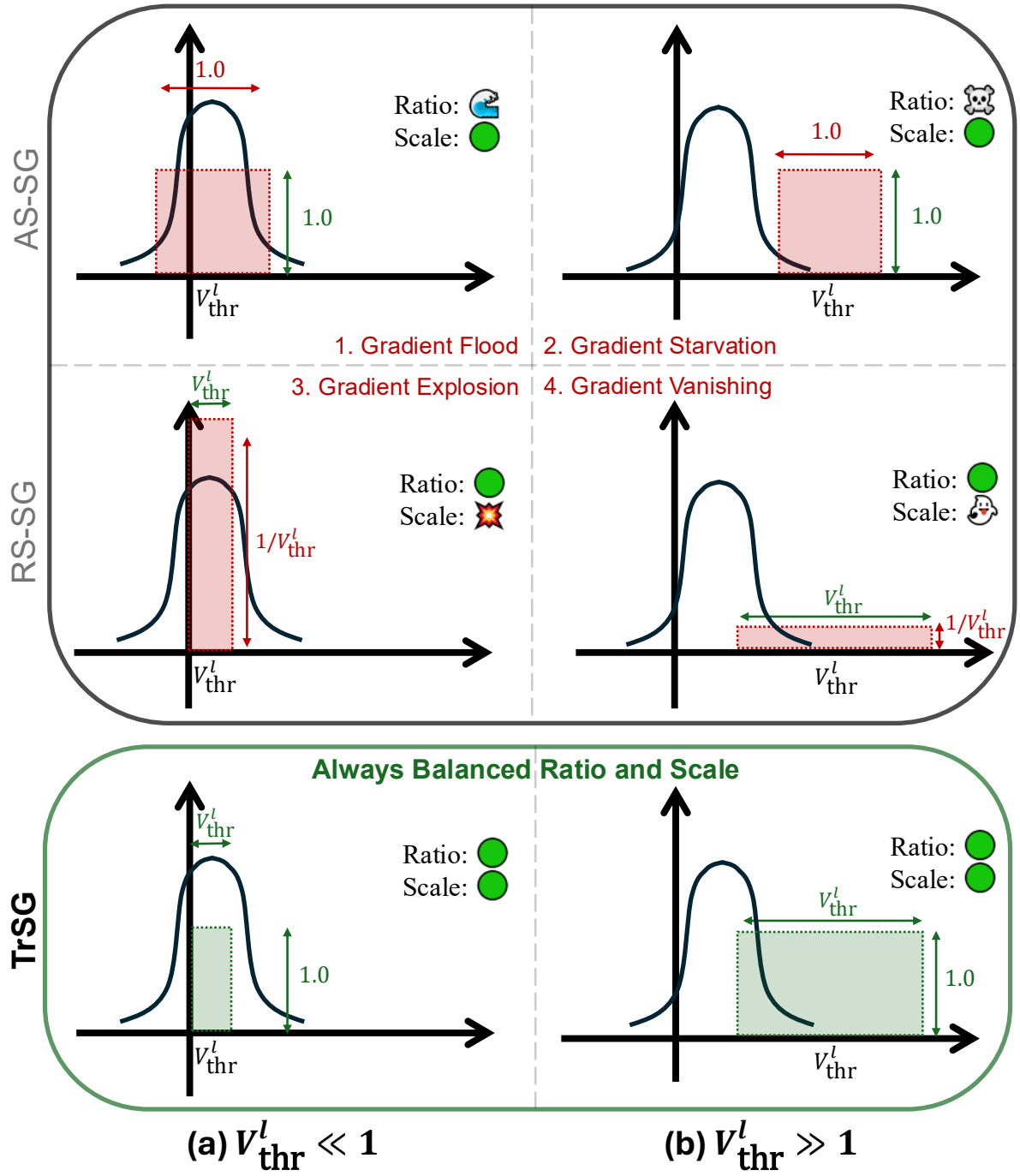


Figure 5.1: **AS-SG vs. RS-SG vs. TrSG with a rectangular surrogate gradient** ($\gamma = 1$)**.** The curve depicts the membrane-potential distribution; colored boxes represent the surrogate-gradient region (horizontal extent: active gradient window; vertical extent: gradient magnitude). Panels (a) and (b) compare small ($V^l_{\text{thr}} \ll 1$) and large ($V^l_{\text{thr}} \gg 1$) thresholds. *AS-SG* uses a fixed window of width γ, leading to *gradient flooding* or *starvation*. *RS-SG* normalizes the window by V^l_{thr} but scales the magnitude as $1/V^l_{\text{thr}}$, causing *explosion* or *vanishing*. **TrSG** multiplies the threshold on the forward path during training, canceling the $1/V^l_{\text{thr}}$ factor and keeping both window and magnitude balanced.

VI. Experiments

We evaluate **MP-Init** and **TrSG** on both static and event-based vision benchmarks: CIFAR-10, CIFAR-100 [40], ImageNet [41], and DVS-CIFAR10 [42], with a variety of backbones (ResNet-19 [38], a 7-layer CNN [15, 16], ResNet-34 [43], SEW-ResNet-34 [44]). Following tdBN [38], batch normalization is applied across both temporal and batch dimensions. Baselines are trained under their original configurations, and we strictly follow their reported preprocessing pipelines for fairness. We avoid strong augmentations such as AutoAugment [45] or Cutout [46] so that the observed gains come solely from our proposed methods. Each configuration is repeated three times and we report mean$\pm$standard deviation.

6.1 Experimental Setup

Architectures. For CIFAR-10/100 [40] and ImageNet [41] we use ResNet-19 [38] and ResNet-34 [43]. On DVS-CIFAR10 [42] we additionally use a 7-layer CNN (64C3-AP2-128C3-128C3-AP2-256C3-256C3-AP2-1024FC-10FC) following [15, 16]. We integrate the temporal dimension into the batch dimension to ensure proper normalization, following tdBN [38].

In MP-Init and TrSG, the running mean of the membrane potential $\mathbb{E}[U[T]]$, the threshold V_{thr}, and the time-decay constant τ are layer-wise parameters. We also experimented with channel-wise versions but observed only a small difference in performance. We initialize $\mathbb{E}[U[T]]$ to 0.0, V_{thr} to 1.0, and τ to 2.0, following common SNN practice [23, 15, 44, 17, 39, 31, 47, 48]; we do not retune these per dataset for fair comparison. The first layer acts as the stem layer that generates spikes from the input, while the final layer does not generate spikes [23]. On CIFAR-10/100 and DVS-CIFAR10, dropout is added before each fully connected layer, consistent with TEBN/TAB [15, 16].

Training configuration.

- **CIFAR-10/100.** ResNet-19 with timesteps $\{2, 4, 6\}$ on a single RTX-3090. SGD with weight decay 5×10^{-4}, CosineAnnealing, initial learning rate 0.02, batch size 64. Standard random-crop, horizontal flip, and normalization.
- **DVS-CIFAR10.** 7-layer CNN and ResNet-19 with timesteps 4 and 10. Same optimizer/scheduler as above; images cropped to 48×48 and horizontally flipped, following [15, 16].
- **ImageNet.** ResNet-34 trained with $T = 6$, evaluated at multiple inference timesteps without fine-tuning. SGD with weight decay 4×10^{-5}, CosineAnnealing starting at 0.10, batch size 512 across

Dataset	Method	Timestep	Accuracy (%)
CIFAR-100	tdBN	4	75.55 ± 0.06
	+ MP-Init		$\mathbf{76.09 \pm 0.04}$
	TEBN	4	75.96 ± 0.68
	+ MP-Init		$\mathbf{76.45 \pm 0.24}$
	TAB	4	76.25 ± 0.49
	+ MP-Init		$\mathbf{77.24 \pm 0.12}$
DVS-CIFAR10	tdBN	10	76.60 ± 0.20
	+ MP-Init		$\mathbf{77.37 \pm 0.38}$
	TEBN	10	75.17 ± 0.50
	+ MP-Init		$\mathbf{76.70 \pm 0.78}$
	TAB	4	76.43 ± 0.25
	+ MP-Init		$\mathbf{76.93 \pm 0.38}$

Table 6.1: Performance gains from MP-Init using ResNet-19 on CIFAR-100 and DVS-CIFAR10.

$8\times$ A100-80GB with synchronized BN. SEW-ResNet-34 follows the same procedure.

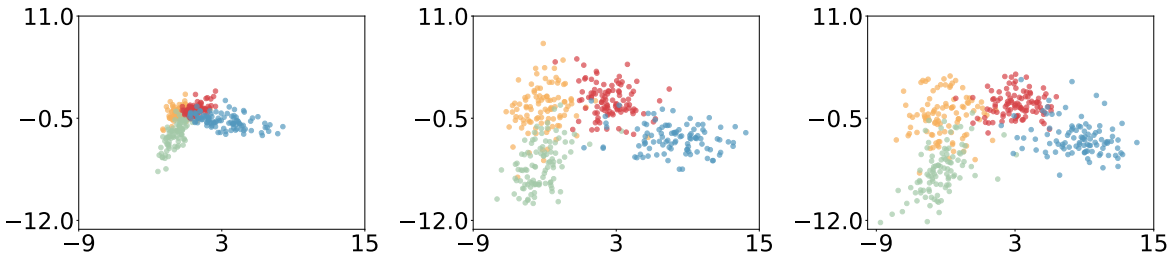
6.2 Effectiveness of MP-Init

We first evaluate whether MP-Init enhances final performance. Table 6.1 reports ResNet-19 accuracies on CIFAR-100 and DVS-CIFAR10 with and without MP-Init across three baselines. We observe consistent gains, not only in tdBN—which does not account for TCS—but also in TCS-aware methods (TEBN, TAB). This confirms that prior approaches address TCS only partially, while MP-Init provides a more fundamental remedy.

6.2.1 Impact of TCS and Role of MP-Init

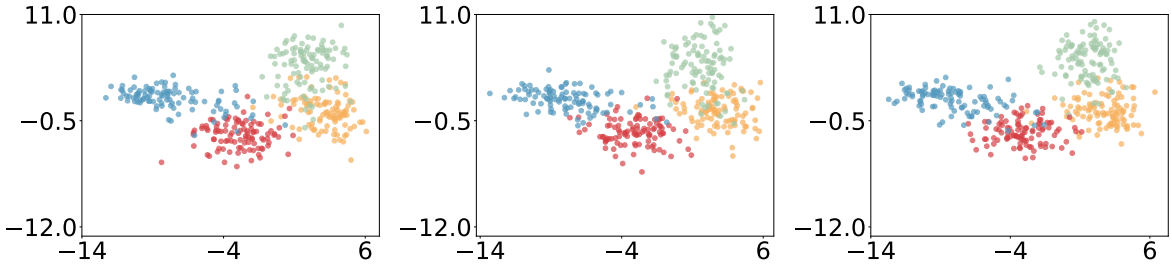
To visualize the effect of TCS we project the final logits of four randomly chosen classes onto two principal components. Figure 6.1 shows that without MP-Init, classes remain entangled in early steps and only gradually separate, reflecting TCS-induced output bias. With MP-Init (Figure 6.2), class separation emerges from the very first timestep.

This is consistent with the firing-rate dynamics in Figure 6.3: MP-Init stabilizes neuron activations across timesteps, whereas the baseline accumulates activity over time. The class-prediction histograms in Figure 6.4 reinforce the picture: without MP-Init, early-step predictions collapse to a single dominant class; with MP-Init, the distribution is balanced from $t = 1$.



(a) $t = 1$ (b) $t = 2$ (c) $t = 4$

Figure 6.1: PCA of logits without MP-Init. Class clusters stay mixed in early timesteps, reflecting strong TCS.



(a) $t = 1$ (b) $t = 2$ (c) $t = 4$

Figure 6.2: PCA of logits with MP-Init. Clear class separation emerges from $t = 1$.

6.2.2 Firing Rate vs. Accuracy

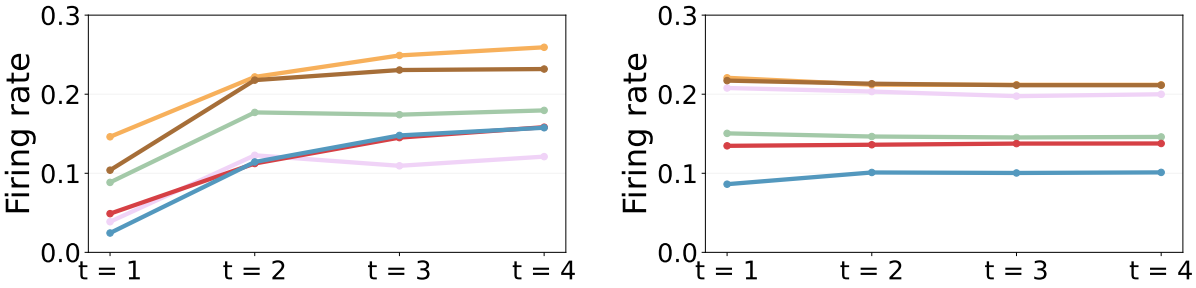
A natural concern is whether MP-Init merely increases firing and thereby harms efficiency. To test this we apply the spike-sparsity regularizer of [49] for ResNet-19 on CIFAR-100 and evaluate performance across firing rates (Figure 6.5). MP-Init does not rely on excessive spiking: it forms a Pareto frontier above the baseline, achieving higher accuracy at matched or lower firing rates. Even at a firing rate of 0.06, MP-Init outperforms all baseline settings—indicating that MP-Init enables more energy-efficient SNN operation in addition to higher accuracy.

6.3 Effectiveness of TrSG

We compare three SG approaches—**AS-SG**, **RS-SG**, and **TrSG**—when training both V_{thr} and τ. To ensure fair evaluation, we use the same training recipes across all methods. To assess initialization sensitivity, we sweep the initial values of V_{thr} and τ.

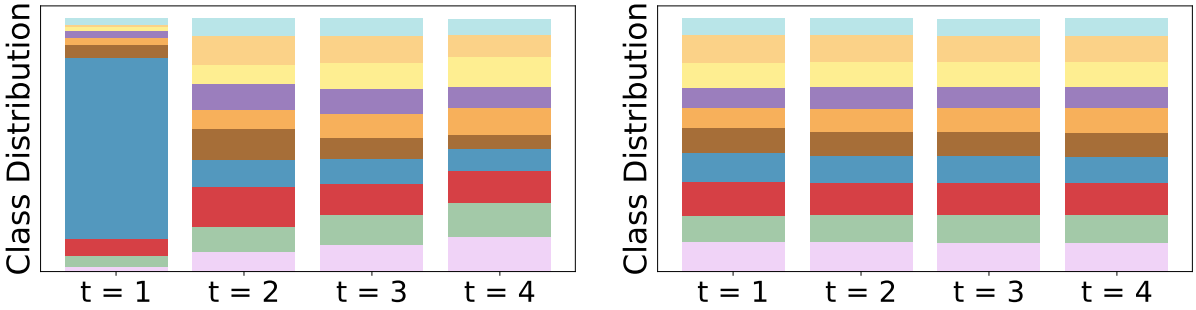
We evaluate each configuration using both *soft* and *hard* reset LIF neurons and two widely adopted surrogate-derivative shapes:

- **Rectangular** [38, 6, 18]: $f'_{sr}(x) = \frac{1}{\gamma}\,\mathbb{1}\left(|x| < \frac{\gamma}{2}\right)$



(a) w/o MP-Init (b) w/ MP-Init

Figure 6.3: Firing rate at each timestep across six layers of ResNet-19 on CIFAR-10.



(a) w/o MP-Init (b) w/ MP-Init

Figure 6.4: Class-prediction distributions over timesteps for ResNet-19 on CIFAR-10.

- **Triangular** [39, 23, 16]: $f'_{sr}(x) = \frac{1}{\gamma^2}\max(0, \gamma - |x|)$

Figure 6.6(a)–(b) (top row) shows final accuracies on CIFAR-100 with ResNet-19 across various initial V_{thr}. Across all conditions, **TrSG** consistently outperforms AS-SG and RS-SG, exhibiting robustness to the initial threshold. The bottom row of Figure 6.6 shows training-loss and validation-accuracy curves: for both reset regimes, TrSG converges faster and reaches higher final accuracy. A similar trend holds when varying initial τ and when using different surrogate shapes (Arctan, Sigmoid).

6.4 Ablation Study

Table 6.2 ablates MP-Init and TrSG on ResNet-19. Both methods independently improve over the baseline; their combination yields the best result, confirming complementary contributions.

Building on these findings—and given their superior quality and stability—we adopt the **rectangular** surrogate function and **soft reset** as the default configuration in subsequent comparisons.

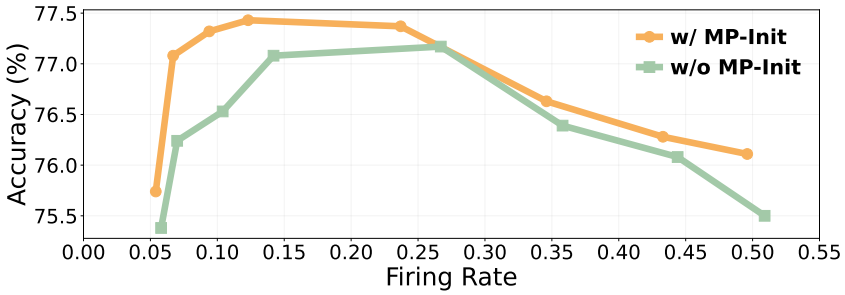


Figure 6.5: Accuracy vs. firing rate. MP-Init forms a strict Pareto frontier.

Dataset	MP-Init	TrSG	Accuracy (%)
CIFAR-100 (T=4)	✗	✗	75.55 ± 0.06
	✓	✗	76.09 ± 0.04
	✗	✓	77.15 ± 0.06
	✓	✓	$\mathbf{77.66 \pm 0.15}$
DVS-CIFAR10 (T=10)	✗	✗	76.60 ± 0.20
	✓	✗	77.37 ± 0.20
	✗	✓	80.83 ± 0.64
	✓	✓	$\mathbf{81.43 \pm 0.40}$

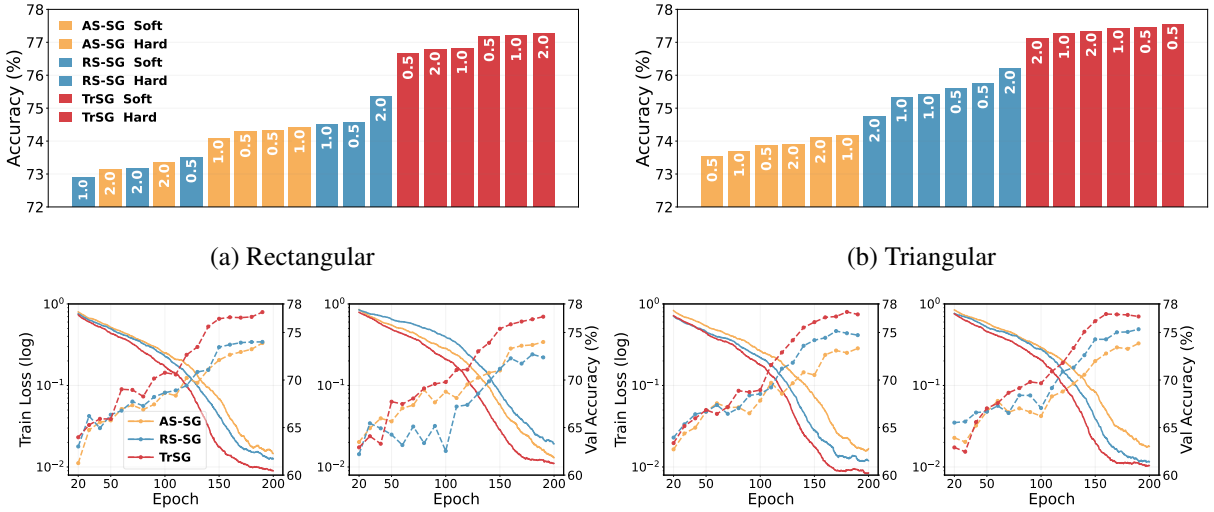
Table 6.2: Ablation of MP-Init and TrSG on ResNet-19.

6.5 Comparison with State-of-the-Art

We compare **MP-Init + TrSG** (referred to as "Ours") against state-of-the-art methods on static datasets (CIFAR-10/100, ImageNet) and the dynamic dataset DVS-CIFAR10, using standard LIF neurons and various backbones. Table 6.3 summarizes the results.

CIFAR-10/100. Ours achieves the best accuracy across all evaluated timesteps. On CIFAR-10 we reach 95.50% at $T = 6$, improving over the previous best (TAB at 94.81%) by 0.69%pt. On CIFAR-100 we obtain 77.91% at $T = 6$, surpassing the strongest baseline (TAB at 76.82%) by 1.09%pt. Our results at fewer timesteps already outperform baselines at larger timesteps, highlighting improved efficiency.

ImageNet. With ResNet-34, our method reaches 68.73% top-1 accuracy at $T = 6$. With SEW-ResNet-34 we further reach 69.67% at $T = 4$, outperforming IMP+LTS [31] (68.90%) and MPS [32] (69.03%). Unlike those approaches, ours adds negligible overhead and does not require neuron-specific extra parameters or complex optimization.



(a) Rectangular (b) Triangular

(c) Rect (soft) (d) Rect (hard) (e) Tri (soft) (f) Tri (hard)

Figure 6.6: **SG approaches under different surrogate shapes (rectangular vs. triangular) and reset types (soft vs. hard).** *Top row:* final CIFAR-100 accuracy (ResNet-19, $T = 4$) for initial $V_{\text{thr}} \in \{0.5, 1.0, 2.0\}$ and $\tau = 2.0$. *Bottom row:* training loss / validation accuracy curves with $\tau = 2.0$, $V_{\text{thr}} = 1.0$. TrSG consistently outperforms AS-SG and RS-SG, demonstrating robustness to initial values.

DVS-CIFAR10. With ResNet-19 at $T = 10$ we obtain 81.43%, exceeding the previous best (RMP-Loss at 76.20%). With a lightweight 7-layer CNN we still reach 79.27% at $T = 4$.

6.6 Extensions to Advanced Architectures and Tasks

We further evaluate MP-Init and TrSG on **Transformer-based SNNs**. On DVS-CIFAR10 (10 timesteps) our method improves SpikeFormer2 from 78.90% to 80.10% and QKFormer from 83.80% to 84.80%. On ImageNet (4 timesteps) SpikingResformer rises from 74.34% to 75.62%, and QKFormer from 78.80% to 79.90% (Table 6.4). The +1.0–1.3%pt gains indicate that MP-Init and TrSG transfer cleanly to advanced Transformer backbones without architectural changes.

Beyond classification, we validate task generality on COCO-2017 detection with EMS-ResNet10 [55]. With identical training configurations (100 epochs) for the baseline and ours, MP-Init + TrSG improves mAP from 0.291 to 0.305 (mAP@0.5) and from 0.138 to 0.147 (mAP@0.5:0.95), as summarized in Table 6.5.

Dataset	Method	Network	Timestep	Accuracy (%)
CIFAR-10	tdBN [38]	ResNet-19	2 / 4 / 6	92.34 / 92.92 / 93.16
	TET [23]		2 / 4 / 6	94.16 / 94.44 / 94.50
	TEBN [15]		2 / 4 / 6	94.57 / 94.70 / 94.71
	TAB [16]		2 / 4 / 6	94.73 / 94.76 / 94.81
	Ours		2 / 4 / 6	**95.05 ± 0.15 / 95.34 ± 0.06 / 95.50 ± 0.16**
CIFAR-100	TET [23]	ResNet-19	2 / 4 / 6	72.87 / 74.47 / 74.72
	TEBN [15]		2 / 4 / 6	75.86 / 76.13 / 76.41
	TAB [16]		2 / 4 / 6	76.31 / 76.81 / 76.82
	Ours		2 / 4 / 6	**76.87 ± 0.16 / 77.66 ± 0.15 / 77.91 ± 0.32**
ImageNet	tdBN [38]	ResNet-34	6	63.72
	Dspike [22]		6	68.19
	TEBN [15]		4	64.29
	TAB [16]		4	67.78
	SEW-ResNet [44]	SEW-ResNet-34	4	67.04
	TET [23]		4	68.00
	TEBN [15]		4	68.28
	IMP+LTS [31]		4	68.90
	MPS [32]		4	69.03
	EAGD [50]		4	68.12
	Ours	ResNet-34	4 / 6	67.15 ± 0.03 / **68.73 ± 0.09**
		SEW-ResNet-34	4	**69.67 ± 0.08**
DVS-CIFAR10	tdBN [38]	ResNet-19	10	67.80
	MPBN [26]		10	74.40 ± 0.20
	RMP-Loss [25]		10	76.20 ± 0.20
	BNTT [14]	7-layer CNN	20	63.20
	TEBN [15]		10	75.10
	TAB [16]		4	76.70
	Ours	ResNet-19	10	**81.43 ± 0.40**
		7-layer CNN	4	**79.27 ± 0.75**

Table 6.3: Comprehensive comparison of our methods on static and dynamic datasets.

6.7 How Does MP-Init Impact Training?

TEBN [15] and TAB [16] have shown that temporal covariate shift indeed occurs, and Chapter IV demonstrated that MP-Init effectively resolves it. Prior studies, however, did not provide direct evidence of *how* TCS impairs training. Inspired by the ensemble interpretation of SNNs in MPS [32], we re-visit this question with a gradient-conflict experiment on ResNet-19 with CIFAR-100. We compute the

Dataset	Architecture	Accuracy (%)
DVS-CIFAR10	Spikformer [51]	80.90
	Spikingformer [52]	81.30
	S-Transformer [53]	80.00
	SpikeFormer2 [54]	78.90
	+ **Ours**	**80.10**
	QKFormer [47]	83.80
	+ **Ours**	**84.80**
ImageNet	Spikformer [51]	70.24
	Spikingformer [52]	72.45
	S-Transformer [53]	72.28
	SpikingResformer [48]	74.34
	+ **Ours**	**75.62**
	QKFormer [47]	78.80
	+ **Ours**	**79.90**

Table 6.4: Transformer-based SNNs with and without our method. DVS-CIFAR10 is evaluated at 10 timesteps; ImageNet at 4 timesteps. Baselines are matched in parameter budget.

Architecture	COCO2017 (mAP@0.5 / mAP@0.5:0.95)
EMS-ResNet10 [55]	0.291 / 0.138
+ **Ours**	**0.305 / 0.147**

Table 6.5: Object detection on COCO-2017 with EMS-ResNet10 ($T = 3$).

gradients applied to a given layer at each timestep and measure the cosine similarity across timesteps.

As shown in Figure 6.7, without MP-Init (left) the gradients at $t = 1$ exhibit strong conflicts with later timesteps, indicating severe inconsistency across the temporal ensemble members. With MP-Init (right), these conflicts are largely alleviated and the overall cosine similarity improves. This provides direct evidence that TCS not only degrades inference consistency but also hinders gradient alignment during training, and confirms that MP-Init effectively stabilizes optimization.

6.8 How Does TrSG Impact Training?

We analyze the gradient flow of ResNet-19 on CIFAR-100 with $T = 4$ for 200 epochs. To isolate the effect of the SG formulation, we *fix* $\tau = 2.0$ and freeze both V_{thr} and τ. We use a rectangular surrogate with $\gamma = 1$ and a soft-reset mechanism.

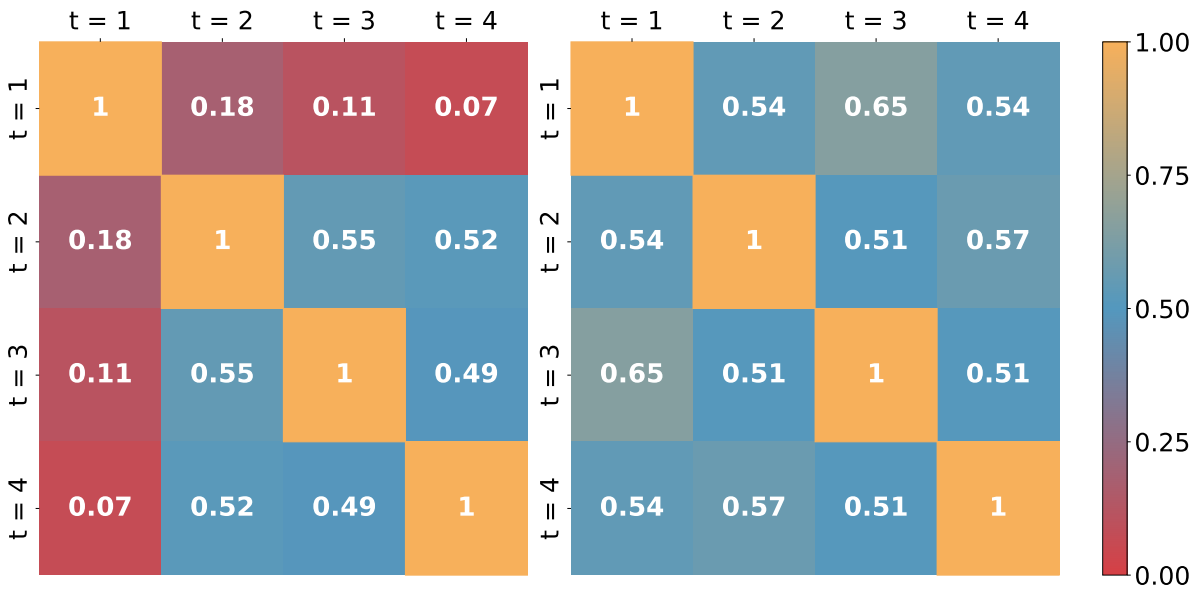


Figure 6.7: **Gradient conflicts across timesteps with and without MP-Init.** Cosine similarity of gradients across timesteps in ResNet-19 on CIFAR-100. *Left:* Without MP-Init, the gradient from $t = 1$ strongly conflicts with later timesteps, reflecting the temporal inconsistency highlighted by MPS [32]. *Right:* With MP-Init, conflicts are largely resolved and similarities improve, indicating that MP-Init aligns temporal ensemble members and stabilizes training.

Stability metrics. We quantify gradient stability with three complementary metrics computed over the convolutional weights of a given layer:

$$\text{AbsStr} = \operatorname*{mean}_i |g_i|, \tag{6.1}$$

$$\text{RatioAG} = \frac{1}{N} \sum_i \mathbb{1}\big(|x_i| < \tfrac{\gamma}{2}\big), \tag{6.2}$$

$$\text{GradCV} = \frac{\text{std}_i(|g_i|)}{\text{mean}_i(|g_i|)}, \tag{6.3}$$

where $g_i = \partial \mathcal{L} / \partial W_i$ and x_i is the surrogate argument. **AbsStr** measures average gradient magnitude—very low values indicate vanishing, very high values indicate explosion, and stable training requires moderate values. **RatioAG** is the fraction of neurons receiving nonzero gradients—too low yields gradient starvation, too high yields gradient flood, and an intermediate range is ideal. **GradCV** captures variability of gradient magnitudes—a low CV (< 1) implies smooth, balanced updates handled by a single learning rate, whereas CV $\gg 1$ means some weights update disproportionately faster than others, destabilizing training. TrSG is designed to maintain moderate AbsStr, healthy RatioAG, and low GradCV simultaneously.

6.8.1 Reference Case: $V_{\text{thr}} = 1.0$

With the widely used threshold $V_{\text{thr}} = 1.0$, all SG variants behave identically (Table 6.6). Although 1.0 is not necessarily optimal, it serves as a sanity check: from epoch 1 to epoch 100 we observe steadily improving yet stable gradients across AS-SG, RS-SG, and TrSG.

Epoch	V_{thr}=1.0	**AbsStr**	**RatioAG (%)**	**GradCV**
1	any SG	0.0122	6.72	0.818
100	any SG	0.0211	18.97	0.571

Table 6.6: Reference case where all SG variants behave identically (ResNet-19, CIFAR-100, $T = 4$, $\tau = 2.0$).

6.8.2 Low Threshold Stress Test: $V_{\text{thr}} = 0.1$

This stress test exposes small-threshold failure modes (Table 6.7). At epoch 1, AS-SG floods (its wide window covers a too-large fraction of the membrane distribution), RS-SG explodes due to the $1/V_{\text{thr}}$ factor (AbsStr is over $1700\times$ larger than TrSG's), while TrSG remains stable. By epoch 100, AS-SG still shows instability with $\mathrm{RatioAG} = 90.20\%$ (extreme flooding), and RS-SG diverges to NaN. TrSG trains smoothly throughout. The accuracy outcomes of Table 6.9 below are consistent with these dynamics.

Epoch	V_{thr}=0.1	**AbsStr**	**RatioAG (%)**	**GradCV**
1	AS-SG	0.2756	31.25	5.01
	RS-SG	7.8191	0.01	37.11
	TrSG	0.0046	3.05	**0.861**
100	AS-SG	0.0145	90.20	1.22
	RS-SG	NaN	0.00	NaN
	TrSG	0.0153	21.45	**0.865**

Table 6.7: Low-threshold stress test (CIFAR-100, $T = 4$, $\tau = 2.0$). RS-SG diverges, AS-SG floods, while TrSG remains stable.

6.8.3 High Threshold Stress Test: $V_{\text{thr}} = 2.0$

For the large threshold $V_{\text{thr}} = 2.0$, the absolute window of AS-SG is too narrow and the $1/V_{\text{thr}}$ factor in RS-SG suppresses signals (Table 6.8). AS-SG starves (its activation ratio collapses to 0% by epoch 100, with literally zero gradient flow), and RS-SG vanishes with high variance ($\mathrm{GradCV} = 2.247$). TrSG preserves stable updates throughout.

Epoch	V_{thr}=2.0	AbsStr	RatioAG (%)	GradCV
1	AS-SG	0.0058	0.78	17.14
	RS-SG	0.0066	1.64	0.639
	TrSG	0.0092	1.62	**0.961**
100	AS-SG	0.0000	0.00	0.000
	RS-SG	0.0021	0.10	2.247
	TrSG	0.0323	6.90	**0.731**

Table 6.8: High-threshold stress test (CIFAR-100, $T = 4$, $\tau = 2.0$). AS-SG starves, RS-SG vanishes, TrSG preserves stable updates.

6.8.4 Accuracy with Frozen Internal Parameters

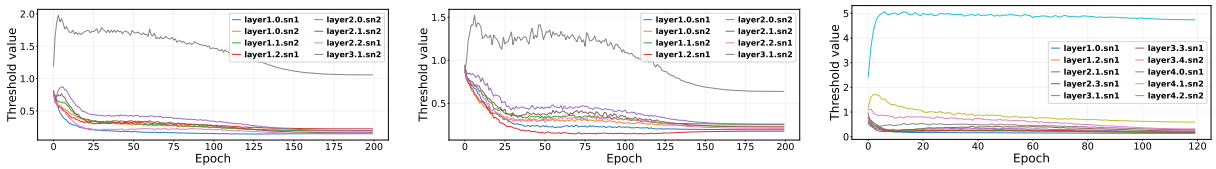
Table 6.9 reports final accuracy when both V_{thr} and τ are *frozen* (so as to isolate the SG effect), with $\tau = 2.0$ and $V_{\text{thr}} \in \{0.1, 0.5, 1.0, 1.5, 2.0\}$. Conventional SGs fail outside a narrow range—exploding at 0.1, starving or vanishing at 2.0—while TrSG remains robust across all thresholds.

Reset	SG	V_{thr}				
		0.1	0.5	1.0	1.5	2.0
Soft	AS-SG	61.59	75.96	—	69.97	18.09
	RS-SG	*div.*	76.12	—	71.43	5.43
	TrSG	**73.99**	**76.82**	**75.56**	**71.97**	**64.27**
Hard	AS-SG	67.71	**76.34**	—	68.42	12.27
	RS-SG	10.36	75.85	—	72.51	52.10
	TrSG	**68.94**	76.33	**75.72**	**73.86**	**68.42**

Table 6.9: Accuracy (%) with fixed $\tau = 2.0$ and *frozen* V_{thr} to isolate SG effects. TrSG remains robust across thresholds, while AS-SG and RS-SG fail outside a narrow range.

6.8.5 Trajectories of Trainable Thresholds

The discussion above raises a natural question: *do real trainings actually visit such extreme thresholds?* The answer is yes. When V_{thr} is trainable, many layers drift toward *lower* thresholds (often $\ll 1$), while late or deep layers move *upward* toward larger values (around ~ 2 on CIFAR-100, and even higher on ImageNet). Figure 6.8 summarizes these trajectories. CIFAR-100 and DVS-CIFAR10 show several early layers dropping below 1 and the last layer increasing, whereas ImageNet exhibits a wider spread with some deep layers rising well beyond 2. These observations confirm that the brittle regimes identified for AS-SG and RS-SG are encountered in practice—precisely where TrSG's threshold-robustness is



(a) ResNet-19, CIFAR-100 (b) ResNet-19, DVS-CIFAR10 (c) ResNet-34, ImageNet

Figure 6.8: **Trajectories of trainable V_{thr} during training.** Early layers tend to settle at lower thresholds, while late or deep layers move toward larger thresholds.

most beneficial.

Implications.

- Tables 6.6 to 6.8 explain *why* TrSG is superior: it maintains moderate AbsStr, healthy RatioAG, and low GradCV—simultaneously avoiding flooding/starvation and explosion/vanishing.
- The threshold trajectories in Figure 6.8 show that real trainings visit the regimes where AS-SG and RS-SG break down, so threshold-robustness is not a corner case but a practical necessity.
- Even when V_{thr} and τ are *not* trained, TrSG is the best choice across thresholds (Table 6.9); using TrSG is strictly beneficial, with no downside in practice.

6.9 Validation on More Surrogate Functions

In Chapter V we introduced TrSG using the rectangular surrogate as the primary example. Section 6.3 extended the experiments to the triangular surrogate. We now demonstrate TrSG's generality by evaluating it with the Arctan-based [17] and Sigmoid-based [10] surrogates while varying the initial threshold V_{thr}.

Arctan surrogate. The derivative of the Arctan-based surrogate is

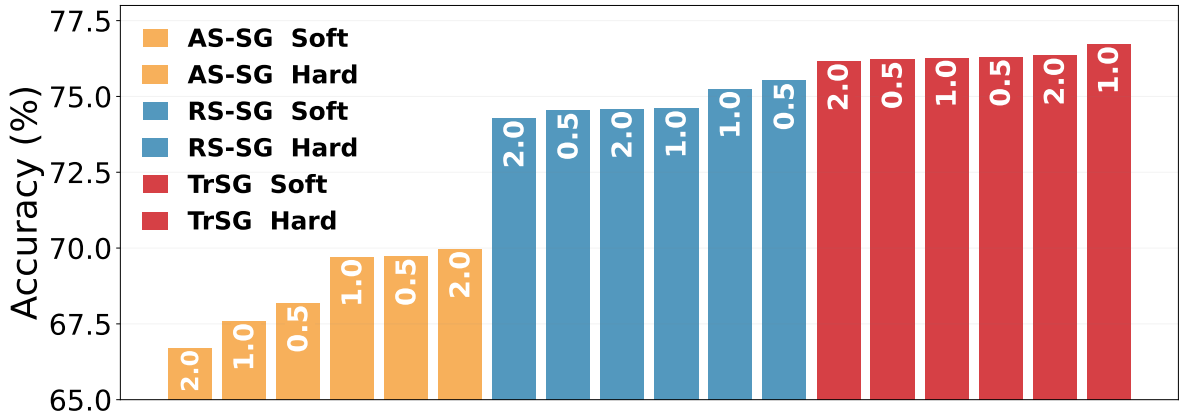
$$\frac{\partial f_{sr}(x)}{\partial x} = \frac{\gamma}{2\big(1+(\frac{\pi}{2}\,\gamma\,x)^2\big)}. \tag{6.4}$$

Sigmoid surrogate. The derivative of the Sigmoid-based surrogate is

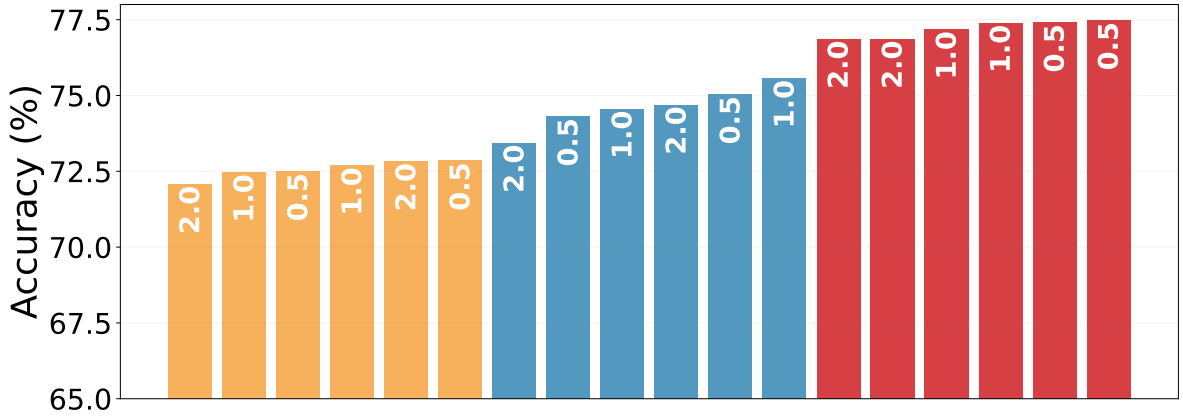
$$\frac{\partial f_{sr}(x)}{\partial x} = \frac{1}{\gamma}\cdot\frac{e^{x/\gamma}}{(1+e^{x/\gamma})^2}. \tag{6.5}$$

Here γ is the steepness parameter. As before, we compare AS-SG, RS-SG, and TrSG under both soft and hard reset.

Figure 6.9 shows that TrSG consistently outperforms AS-SG and RS-SG under both Arctan and Sigmoid surrogates, regardless of the initial threshold. Together with the rectangular and triangular results of Section 6.3, this confirms that TrSG's threshold-robustness extends across all common surrogate-gradient shapes.



(a) Arctan surrogate



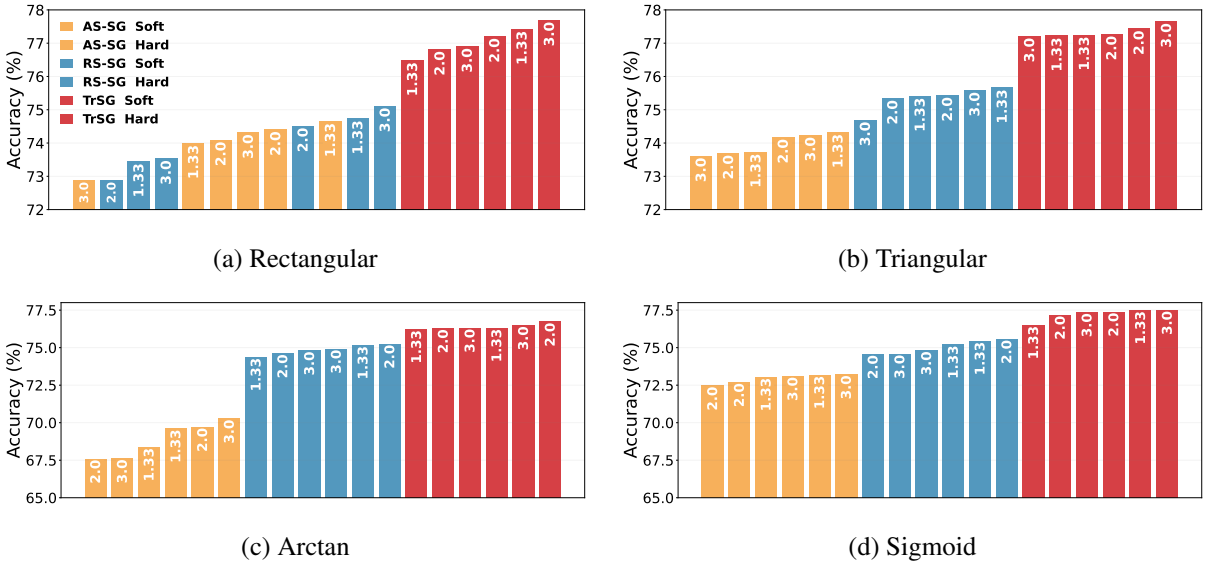
(b) Sigmoid surrogate

Figure 6.9: **Accuracy under different initial $V_{\text{thr}} \in \{0.5, 1.0, 2.0\}$ for Arctan and Sigmoid surrogates ($\tau = 2.0$).** We compare AS-SG, RS-SG, and TrSG on ResNet-19 (CIFAR-100, $T = 4$), evaluating both soft- and hard-reset LIF neurons.

6.10 Varying Initial τ with a Fixed Initial V_{thr}

To complement the experiments in Sections 6.3 and 6.9 (which varied V_{thr}), we now fix $V_{\text{thr}} = 1.0$ and vary the initial leakage constant $\tau \in \{1.33, 2.0, 3.0\}$. Figure 6.10 reports accuracy across all four surrogate shapes—rectangular, triangular, Arctan, and Sigmoid—each evaluated with both soft and hard resets. Across every configuration, TrSG consistently outperforms AS-SG and RS-SG, maintaining high

accuracy regardless of the initial τ and reaffirming the robustness observed under varying V_{thr}.



(a) Rectangular

(b) Triangular

(c) Arctan

(d) Sigmoid

Figure 6.10: **Accuracy under different initial $\tau \in \{1.33, 2.0, 3.0\}$ with $V_{\text{thr}} = 1.0$, across all four surrogate shapes.** (a) Rectangular, (b) triangular, (c) Arctan, (d) Sigmoid—each evaluated with both soft and hard resets. ResNet-19 on CIFAR-100, $T = 4$. TrSG consistently outperforms AS-SG and RS-SG in every panel.

6.11 Impact of Trainable Parameters on Accuracy

To analyze the impact of training the threshold V_{thr} and the leakage constant τ, we run an ablation study (Table 6.10). Training only τ slightly degrades accuracy compared with fixed parameters. Training only V_{thr} yields a noticeable improvement. Training both jointly produces the best result (77.22%). Although τ alone is not beneficial, its effect becomes synergistic when combined with V_{thr} training: the network jointly adapts the integration time and the firing threshold to balance convergence speed and stability.

τ trainable	V_{thr} trainable	Accuracy (%)
✗	✗	75.56
✓	✗	74.61
✗	✓	76.98
✓	✓	**77.22**

Table 6.10: Ablation of trainable internal LIF parameters on ResNet-19, CIFAR-100, $T = 4$. Initial values are $V_{\text{thr}} = 1.0$ and $\tau = 2.0$.

6.12 Ablation on Advanced Architecture

We further ablate the surrogate-gradient choice on a large-scale Transformer SNN, QKFormer [47], using the official training protocol while keeping all settings fixed and changing only the SG variant (Table 6.11). With MP-Init enabled across the board, TrSG is the only variant that reliably improves over the baseline ($+1.10$ %pt). AS-SG underperforms (-1.71 %pt) and RS-SG essentially matches or slightly underperforms (-0.10 %pt). These results indicate that even on advanced Transformer-based SNNs, TrSG provides consistent benefits, and that conventional SGs can in fact *degrade* performance—underscoring TrSG as the safer and superior choice for modern large-scale SNN architectures.

Method	**Top-1 (%)**
QKFormer (HST-10-384)	78.80
+ AS-SG	77.09
+ RS-SG	78.70
+ **TrSG**	**79.90**

Table 6.11: Ablation of the surrogate-gradient choice on QKFormer (HST-10-384), ImageNet, $T = 4$. All runs follow the official training recipe with only the SG variant differing. MP-Init is enabled in every case.

6.13 Accuracy vs. Firing Rate under TrSG

Section 6.2.2 showed that MP-Init forms a Pareto front over firing rate. A natural question is whether training the internal parameters V_{thr} and τ with TrSG merely increases spike counts to boost accuracy. To decouple these effects, we sweep the same sparsity regularization loss [49] as before and compare *Training* (TrSG updates V_{thr} and τ) against *No Training* (both frozen).

As shown in Figure 6.11, the Training curve strictly dominates the No-Training curve at all firing rates, indicating that TrSG's improvement stems from well-adapted neuron dynamics rather than excessive spiking. This is consistent with the threshold trajectories of Figure 6.8, where layers evolve toward operating points that favor stable, informative spikes.

6.14 Energy Cost

Since the firing rate tightly correlates with accumulate operations (ACs)—and therefore energy—the analyses in Section 6.2.2 and Section 6.13 can be viewed as an energy–accuracy trade-off. To quantify this, we measure ACs and multiply–accumulate operations (MACs) and compute the corresponding energy following [56]; Table 6.12 reports the result. At similar firing rates (and hence similar energy cost),

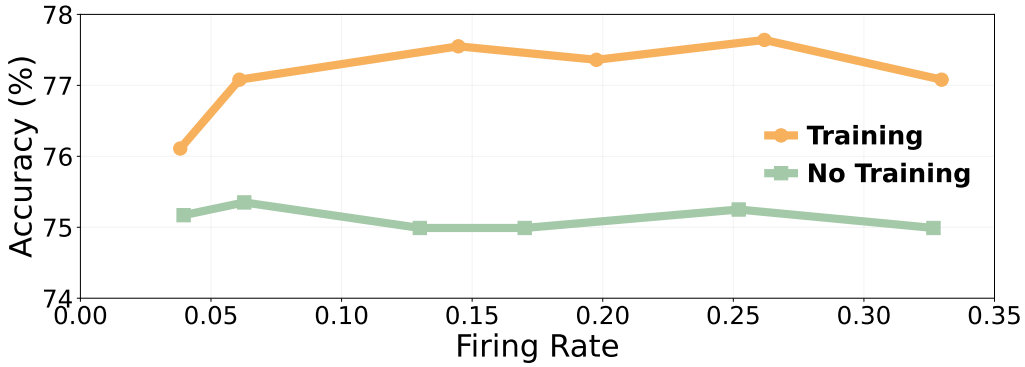


Figure 6.11: **Accuracy vs. firing rate under TrSG.** We sweep a sparsity loss [49] to match global firing rates and compare *Training* (learn V_{thr} and τ with TrSG) against *No Training* (both frozen).

training the internal neuron parameters with TrSG yields consistently higher accuracy. In short, TrSG improves accuracy at nearly the same energy cost—the gains come from better adaptation of V_{thr} and τ, not from increasing spike counts.

MP-Init	TrSG	ACs / MACs (G)	Energy (mJ)	Acc (%)
✗	✗	1.03 / 0.14	1.57	75.58
✓	✗	1.01 / 0.14	1.57	76.20
✗	✓	1.13 / 0.14	1.68	77.13
✓	✓	1.12 / 0.14	1.67	**77.68**

Table 6.12: **Energy at comparable firing rates.** Accumulate operations (ACs), multiply–accumulate operations (MACs), and computed energy cost. ResNet-19, CIFAR-100, $T = 4$.

6.15 Final Values of Neuron Parameters

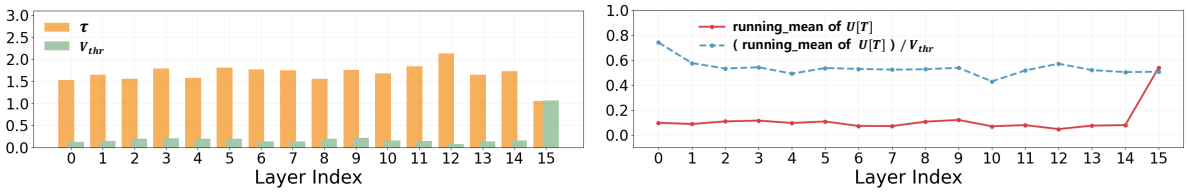
In addition to learning the network weights, our approach optimizes the neuron parameters V_{thr} and τ, while MP-Init tracks an internal running mean of the membrane potential at the last timestep, $U[T]$. To investigate how these three quantities evolve through training, we visualize their final layer-wise values for both CIFAR-100 and DVS-CIFAR10 in Figure 6.12, considering soft and hard resets.

Running mean of $U[T]$. The running mean of $U[T]$ typically settles at about half of V_{thr} in the soft-reset case (Figures 6.12a and 6.12b) and roughly a quarter of V_{thr} for hard-reset neurons (Figures 6.12c and 6.12d). This indicates that the average post-synaptic potential remains below the learned threshold in most layers yet remains sufficiently high to generate spikes when needed.

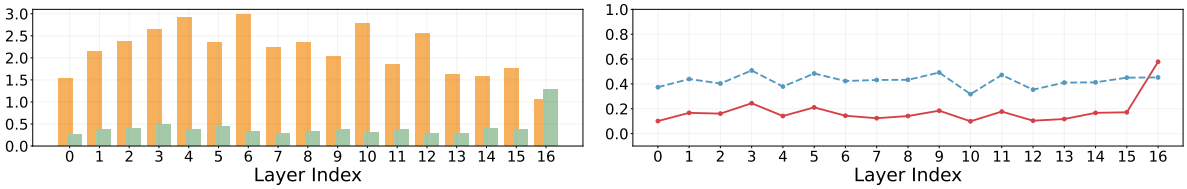
Threshold V_{thr}. Across both datasets and reset types, the learned V_{thr} commonly falls below 0.5 in earlier and intermediate layers, suggesting that moderate thresholds facilitate stable spiking activity.

Decay constant τ. We observe a consistent trend of τ converging around 1.5–2.0 in most layers for CIFAR-100, while on DVS-CIFAR10, τ tends to reach larger values (e.g., near 2.5–3.0). This implies that dynamic event-driven data benefit from neurons accumulating input more slowly, whereas CIFAR-100's static images favor a shorter effective integration time.

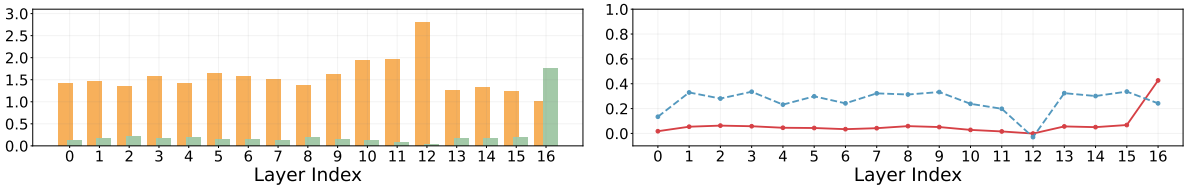
Last-layer behavior. Interestingly, in the final layer on both datasets, τ commonly drops close to 1, effectively making it a binary layer. The layer may act as a gating mechanism, helping the model classify features decisively without excessive temporal integration.



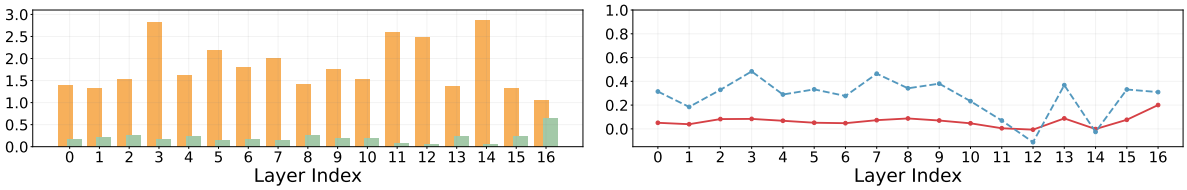
(a) CIFAR-100 (soft reset)



(b) DVS-CIFAR10 (soft reset)



(c) CIFAR-100 (hard reset)



(d) DVS-CIFAR10 (hard reset)

Figure 6.12: Final values of V_{thr}, τ, and the running mean of $U[T]$ after training. ResNet-19 on CIFAR-100 ($T = 4$) and DVS-CIFAR10 ($T = 10$).

6.16 Discussion

The empirical results corroborate the analytical claims of Chapters IV and V: MP-Init removes the residual TCS that prior normalization-based remedies miss, and TrSG removes the threshold-induced gradient pathologies that prior surrogate-gradient designs suffer from. Their effects are complementary—as evidenced by the 2–5 %pt cumulative gain on DVS-CIFAR10—and they generalize cleanly across architectures, datasets, and downstream tasks while preserving the standard LIF inference pipeline. The stability metrics in Section 6.8, the threshold trajectories in Section 6.8.5, and the converged neuron parameters in Section 6.15 together provide a coherent narrative: the gains stem from well-behaved gradient flow and well-adapted neuron dynamics, not from spurious effects such as inflated firing rates.

VII. Conclusion

7.1 Summary of Contributions

This thesis presented two complementary methods for stabilizing the direct training of Spiking Neural Networks.

MP-Init (Chapter IV) traced temporal covariate shift to its root cause—the membrane potential—and proved that the layer-wise membrane potential converges geometrically to a unique stationary distribution under mild conditions. Building on this convergence theorem, MP-Init initializes each layer's membrane potential to a per-layer running mean that approximates the stationary mean. The method eliminates TCS while preserving the standard LIF inference pipeline and adding negligible parameter or computational overhead.

TrSG (Chapter V) classified existing surrogate-gradient methods into Absolute-Scale (AS-SG) and Relative-Scale (RS-SG) families, and showed how each becomes unstable when the threshold voltage is trainable—producing either gradient flooding/starvation or gradient explosion/vanishing. TrSG combines the relative-scale active window with a threshold-multiplied forward path, yielding gradients whose magnitudes are invariant to the threshold scale. Inference reverts to standard binary spikes through a simple weight rescaling, ensuring full hardware compatibility.

Empirical validation (Chapter VI) on CIFAR-10/100, ImageNet, and DVS-CIFAR10 showed that the combination of MP-Init and TrSG achieves state-of-the-art accuracy across both static and dynamic benchmarks. The methods further transfer cleanly to Transformer-based SNN backbones and to event-based object detection, indicating broad applicability beyond classification.

7.2 Limitations

Despite the gains demonstrated in this thesis, several limitations remain.

First, MP-Init mitigates TCS efficiently but uses the simplest possible approximation of the stationary distribution—a per-layer constant mean. Richer parameterizations (e.g., per-channel or distribution-shape-aware initialization) could yield further improvement, at the cost of additional parameters; we did not explore these in depth.

Second, training the internal LIF parameters V_{thr} and τ may be incompatible with neuromorphic hardware that fixes these to constants in silicon. While inference under TrSG reverts to standard LIF dynamics by absorbing V_{thr} into adjacent weights, hardware that does not support per-layer τ would still

require quantization or rounding of the trained values.

Third, the convergence theorem in Chapter IV assumes i.i.d. presynaptic inputs; while we provided empirical support (Ljung–Box tests, bounded $|U|$, geometric TV decay) for trained networks, the assumption may be violated in untrained or pathological networks. A complete characterization of the regime in which the assumption holds is left for future work.

7.3 Future Directions

Several directions seem promising for future research.

Hardware-aware adaptation. Adapting MP-Init and TrSG for hardware with constrained neuron parameters—through quantization-aware training, distillation to fixed-parameter neurons, or hybrid analog-digital schemes—would broaden deployment options.

Beyond LIF. The principles behind MP-Init (initialization at the stationary distribution) and TrSG (threshold-invariant gradient) are not specific to LIF neurons. Extending them to alternative spiking models (e.g., adaptive LIF, Izhikevich) is a natural next step.

Theoretical refinement. Tighter convergence rates, removing the i.i.d. assumption, and analyzing the joint dynamics of τ and V_{thr} during training are open theoretical questions.

Scaling further. Combining MP-Init and TrSG with very large SNN architectures—including SNN versions of vision transformers and modern foundation models—remains an open empirical frontier.

7.4 Closing Remarks

Spiking neural networks offer a promising path toward energy-efficient AI, but their adoption hinges on closing the accuracy gap with conventional deep networks while preserving hardware compatibility. By identifying and resolving two long-standing instabilities of direct SNN training—temporal covariate shift in the membrane potential, and gradient pathology of learnable thresholds—this thesis takes a concrete step in that direction.

요 약 문

본 학위논문은 스파이킹 신경망(Spiking Neural Network, SNN)의 직접 학습에서 발생하는 두 가지 핵심 난제, 즉 시간적 공변량 변화(Temporal Covariate Shift, TCS)와 임계전압 학습 시의 기울기 불안정성을 해결하기 위한 두 가지 기법을 제안한다.

첫 번째 기여인 **MP-Init**(Membrane Potential Initialization)은 TCS의 근본 원인이 활성화가 아닌 LIF 뉴런의 막전위(membrane potential)에 있음을 이론적·실증적으로 규명한다. 본 논문은 이산 시간 LIF 모델에서 막전위가 마르코프 연쇄를 형성하며 두블린(Doeblin) 미노라이제이션 조건 아래 유일한 정상분포로 기하적으로 수렴함을 증명하고, 학습 동안 마지막 시점의 평균을 층(layer)별 누적 평균으로 추정하여 매 배치 시작 시 막전위를 해당 평균으로 초기화한다. 이로써 TCS가 본질적으로 해소되며, 추가 파라미터가 거의 없고 표준 LIF 추론 동작을 보존하므로 뉴로모픽 하드웨어와 완전히 호환된다.

두 번째 기여인 **TrSG**(Threshold-Robust Surrogate Gradient)는 임계전압 V_{thr}이 학습 가능한 파라미터일 때 기존 서러게이트 기울기가 보이는 임계값 스케일 의존성을 처음으로 분석하고 해결한다. 본 논문은 기존 방법을 절대 스케일(AS-SG)과 상대 스케일(RS-SG)로 분류하고, 각각이 *기울기 범람/고갈* 또는 *기울기 폭발/소실* 문제를 일으킴을 보였다. TrSG는 상대 스케일의 활성 영역을 유지하면서 순방향 경로에서 출력에 임계전압을 곱해 기울기 크기에서 $1/V_{\text{thr}}$ 인자를 상쇄시킨다. 추론 시에는 단순 가중치 재조정으로 이진 스파이크 출력을 회복한다.

CIFAR-10/100, ImageNet, DVS-CIFAR10 데이터셋에서의 실험 결과, MP-Init과 TrSG의 결합은 표준 LIF 뉴런과 다양한 백본에서 일관된 최첨단 성능을 달성하였다. Transformer 기반 SNN과 이벤트 기반 객체 검출 과제에도 변형 없이 적용 가능하여 분류 과제를 넘어 폭넓게 효과적임을 확인하였다.

References

[1] Yongqiang Cao, Yang Chen, and Deepak Khosla. Spiking deep convolutional neural networks for energy-efficient object recognition. *International Journal of Computer Vision*, 113(1):54–66, 2015.

[2] Steven K. Esser, Paul A. Merolla, John V. Arthur, Andrew S. Cassidy, Rathinakumar Appuswamy, Alexander Andreopoulos, David J. Berg, Jeffrey L. McKinstry, Timothy Melano, Davis R. Barch, Carmelo Di Nolfo, Pallab Datta, Arnon Amir, Brian Taba, Myron D. Flickner, and Dharmendra S. Modha. Convolutional networks for fast, energy-efficient neuromorphic computing. *Proceedings of the National Academy of Sciences of the United States of America*, 113:11441–11446, 10 2016.

[3] Emre O. Neftci. Data and power efficient intelligence with neuromorphic learning machines. *iScience*, 5:52–68, 2018.

[4] Kaushik Roy, Akhilesh Jaiswal, and Priyadarshini Panda. Towards spike-based machine intelligence with neuromorphic computing. *Nature 2019 575:7784*, 575:607–617, 11 2019.

[5] Benjamin Cramer, Sebastian Billaudelle, Simeon Kanya, Aron Leibfried, Andreas Grübl, Vitali Karasenko, Christian Pehle, Korbinian Schreiber, Yannik Stradmann, Johannes Weis, Johannes Schemmel, and Friedemann Zenke. Surrogate gradients for analog neuromorphic computing. *Proceedings of the National Academy of Sciences of the United States of America*, 119:e2109194119, 1 2022.

[6] Nitin Rathi and Kaushik Roy. Diet-snn: A low-latency spiking neural network with direct input encoding and leakage and threshold optimization. *IEEE Transactions on Neural Networks and Learning Systems*, 34(6):3174–3182, June 2023.

[7] Paul A. Merolla, John V. Arthur, Rodrigo Alvarez-Icaza, Andrew S. Cassidy, Jun Sawada, Filipp Akopyan, Bryan L. Jackson, Nabil Imam, Chen Guo, Yutaka Nakamura, Bernard Brezzo, Ivan Vo, Steven K. Esser, Rathinakumar Appuswamy, Brian Taba, Arnon Amir, Myron D. Flickner, William P. Risk, Rajit Manohar, and Dharmendra S. Modha. A million spiking-neuron integrated circuit with a scalable communication network and interface. *Science*, 345:668–673, 8 2014.

[8] Filipp Akopyan, Jun Sawada, Andrew Cassidy, Rodrigo Alvarez-Icaza, John Arthur, Paul Merolla, Nabil Imam, Yutaka Nakamura, Pallab Datta, Gi Joon Nam, Brian Taba, Michael Beakes, Bernard Brezzo, Jente B. Kuang, Rajit Manohar, William P. Risk, Bryan Jackson, and Dharmendra S.

Modha. Truenorth: Design and tool flow of a 65 mw 1 million neuron programmable neurosynaptic chip. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 34:1537–1557, 10 2015.

[9] Mike Davies, Narayan Srinivasa, Tsung Han Lin, Gautham Chinya, Yongqiang Cao, Sri Harsha Choday, Georgios Dimou, Prasad Joshi, Nabil Imam, Shweta Jain, Yuyun Liao, Chit Kwan Lin, Andrew Lines, Ruokun Liu, Deepak Mathaikutty, Steven McCoy, Arnab Paul, Jonathan Tse, Guruguhanathan Venkataramanan, Yi Hsin Weng, Andreas Wild, Yoonseok Yang, and Hong Wang. Loihi: A neuromorphic manycore processor with on-chip learning. *IEEE Micro*, 38:82–99, 1 2018.

[10] Yujie Wu, Lei Deng, Guoqi Li, Jun Zhu, and Luping Shi. Spatio-temporal backpropagation for training high-performance spiking neural networks. *Frontiers in Neuroscience*, 12, 2018.

[11] Yujie Wu, Lei Deng, Guoqi Li, Jun Zhu, Yuan Xie, and Luping Shi. Direct training for spiking neural networks: Faster, larger, better. In *Proceedings of the AAAI conference on artificial intelligence*, volume 33, pages 1311–1318, Jul. 2019.

[12] Yufei Guo, Xuhui Huang, and Zhe Ma. Direct learning-based deep spiking neural networks: a review. *Frontiers in Neuroscience*, 17:1209795, 6 2023.

[13] Chenlin Zhou, Han Zhang, Liutao Yu, Yumin Ye, Zhaokun Zhou, Liwei Huang, Zhengyu Ma, Xiaopeng Fan, Huihui Zhou, and Yonghong Tian. Direct training high-performance deep spiking neural networks: a review of theories and methods. *Frontiers in Neuroscience*, 18:1383844, 7 2024.

[14] Youngeun Kim and Priyadarshini Panda. Revisiting batch normalization for training low-latency deep spiking neural networks from scratch. *Frontiers in Neuroscience*, 15, 10 2020.

[15] Chaoteng Duan, Jianhao Ding, Shiyan Chen, Zhaofei Yu, and Tiejun Huang. Temporal effective batch normalization in spiking neural networks. In *Advances in Neural Information Processing Systems*, volume 35, pages 34377–34390, 12 2022.

[16] Haiyan Jiang, Vincent Zoonekynd, Giulia De Masi, Bin Gu, and Huan Xiong. TAB: Temporal accumulated batch normalization in spiking neural networks. In *The Twelfth International Conference on Learning Representations*, 2024.

[17] W. Fang, Z. Yu, Y. Chen, T. Masquelier, T. Huang, and Y. Tian. Incorporating learnable membrane time constant to enhance learning of spiking neural networks. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 2641–2651, Los Alamitos, CA, USA, oct 2021. IEEE Computer Society.

[18] SIQI WANG, Tee Hiang Cheng, and Meng-Hiot Lim. Ltmd: Learning improvement of spiking neural networks with learnable thresholding neurons and moderate dropout. In S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, editors, *Advances in neural information processing systems*, volume 35, pages 28350–28362. Curran Associates, Inc., 2022.

[19] Minsu Yoo, Yoon Sil Yang, Jong Cheol Rah, and Joon Ho Choi. Different resting membrane potentials in posterior parietal cortex and prefrontal cortex in the view of recurrent synaptic strengths and neural network dynamics. *Frontiers in Cellular Neuroscience*, 17:1153970, 2023.

[20] Jun Haeng Lee, Tobi Delbruck, and Michael Pfeiffer. Training deep spiking neural networks using backpropagation. *Frontiers in Neuroscience*, 10, 2016.

[21] Friedemann Zenke and Tim P. Vogels. The remarkable robustness of surrogate gradient learning for instilling complex function in spiking neural networks. *Neural Computation*, 33(4):899–925, 03 2021.

[22] Yuhang Li, Yufei Guo, Shanghang Zhang, Shikuang Deng, Yongqing Hai, and Shi Gu. Differentiable spike: Rethinking gradient-descent for training spiking neural networks. In M. Ranzato, A. Beygelzimer, Y. Dauphin, P.S. Liang, and J. Wortman Vaughan, editors, *Advances in Neural Information Processing Systems*, volume 34, pages 23426–23439. Curran Associates, Inc., 2021.

[23] Shikuang Deng, Yuhang Li, Shanghang Zhang, and Shi Gu. Temporal efficient training of spiking neural network via gradient re-weighting. *arXiv*, 2022.

[24] Yufei Guo, Xinyi Tong, Yuanpei Chen, Liwen Zhang, Xiaode Liu, Zhe Ma, and Xuhui Huang. Recdis-snn: Rectifying membrane potential distribution for directly training spiking neural networks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 326–335, June 2022.

[25] Yufei Guo, Xiaode Liu, Yuanpei Chen, Liwen Zhang, Weihang Peng, Yuhan Zhang, Xuhui Huang, and Zhe Ma. Rmp-loss: Regularizing membrane potential distribution for spiking neural networks. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 08 2023.

[26] Y. Guo, Y. Zhang, Y. Chen, W. Peng, X. Liu, L. Zhang, X. Huang, and Z. Ma. Membrane potential batch normalization for spiking neural networks. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 19363–19373, Los Alamitos, CA, USA, oct 2023. IEEE Computer Society.

[27] Wei Fang, Zhaofei Yu, Zhaokun Zhou, Ding Chen, Yanqi Chen, Zhengyu Ma, Timothée Masquelier, and Yonghong Tian. Parallel spiking neurons with high efficiency and ability to learn long-term dependencies. *Advances in Neural Information Processing Systems*, 4 2023.

[28] Xingting Yao, Fanrong Li, Zitao Mo, and Jian Cheng. GLIF: A unified gated leaky integrate-and-fire neuron for spiking neural networks. In *Advances in Neural Information Processing Systems*, 2022.

[29] Yulong Huang, Xiaopeng Lin, Hongwei Ren, Haotian Fu, Yue Zhou, Zunchang Liu, Biao Pan, and Bojun Cheng. Clif: Complementary leaky integrate-and-fire neuron for spiking neural networks. *Proceedings of Machine Learning Research*, 235:19949–19972, 2 2024.

[30] Eric Hunsberger and Chris Eliasmith. Spiking deep networks with lif neurons. *arXiv*, 2015.

[31] Hangchi Shen, Qian Zheng, Huamin Wang, and Gang Pan. Rethinking the membrane dynamics and optimization objectives of spiking neural networks. In A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang, editors, *Advances in Neural Information Processing Systems*, volume 37, pages 92697–92720. Curran Associates, Inc., 2024.

[32] Yongqi Ding, Lin Zuo, Mengmeng Jing, Pei He, and Hanpu Deng. Rethinking spiking neural networks from an ensemble learning perspective. In *ICLR*, 2025.

[33] Jason Kamran Eshraghian, Max Ward, Emre Neftci, Xinxin Wang, Gregor Lenz, Girish Dwivedi, Mohammed Bennamoun, Doo Seok Jeong, and Wei D. Lu. Training spiking neural networks using lessons from deep learning. *CoRR*, abs/2109.12894, 2021.

[34] Bodo Rueckauer, Iulia-Alexandra Lungu, Yuhuang Hu, and Michael Pfeiffer. Theory and tools for the conversion of analog to spiking convolutional neural networks. *arXiv*, 2016.

[35] Ruokai Yin, Abhishek Moitra, Abhiroop Bhattacharjee, Youngeun Kim, and Priyadarshini Panda. Sata: Sparsity-aware training accelerator for spiking neural networks. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 42:1926–1938, 4 2022.

[36] Grégory Dumont and Pierre Gabriel. The mean-field equation of a leaky integrate-and-fire neural network: measure solutions and steady states. *Nonlinearity*, 33:6381–6420, 10 2017.

[37] Jeffrey Rosenthal. Minorization conditions and convergence rates for markov chain monte carlo. *Journal of the American Statistical Association*, 90:558–566, 1995.

[38] Hanle Zheng, Yujie Wu, Lei Deng, Yifan Hu, and Guoqi Li. Going deeper with directly-trained larger spiking neural networks. In *Proceedings of the AAAI conference on artificial intelligence*, volume 35, pages 11062–11070, May 2021.

[39] Qingyan Meng, Mingqing Xiao, Shen Yan, Yisen Wang, Zhouchen Lin, and Zhi-Quan Luo. Towards memory- and time-efficient backpropagation for training spiking neural networks. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2 2023.

[40] Alex Krizhevsky, Geoffrey Hinton, et al. Learning multiple layers of features from tiny images, 2009.

[41] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 248–255, June 2009.

[42] Hongmin Li, Hanchao Liu, Xiangyang Ji, Guoqi Li, and Luping Shi. Cifar10-dvs: An event-stream dataset for object classification. *Frontiers in Neuroscience*, 11, 2017.

[43] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 770–778. IEEE, Jun 2016.

[44] Wei Fang, Zhaofei Yu, Yanqi Chen, Tiejun Huang, Timothée Masquelier, and Yonghong Tian. Deep residual learning in spiking neural networks. In M. Ranzato, A. Beygelzimer, Y. Dauphin, P.S. Liang, and J. Wortman Vaughan, editors, *Advances in neural information processing systems*, volume 34, pages 21056–21069. Curran Associates, Inc., 2021.

[45] Ekin D. Cubuk, Barret Zoph, Dandelion Mane, Vijay Vasudevan, and Quoc V. Le. Autoaugment: Learning augmentation policies from data. In *Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition*, pages 113–123, 5 2018.

[46] Terrance DeVries and Graham W. Taylor. Improved regularization of convolutional neural networks with cutout, 2017.

[47] Chenlin Zhou, Han Zhang, Zhaokun Zhou, Liutao Yu, Liwei Huang, Xiaopeng Fan, Li Yuan, Zhengyu Ma, Huihui Zhou, and Yonghong Tian. Qkformer: Hierarchical spiking transformer using q-k attention. In *Advances in Neural Information Processing Systems*, 3 2024.

[48] Xinyu Shi, Zecheng Hao, and Zhaofei Yu. Spikingresformer: Bridging resnet and vision transformer in spiking neural networks. In *Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition*, 2024.

[49] Yulong Yan, Haoming Chu, Yi Jin, Yuxiang Huan, Zhuo Zou, and Lirong Zheng. Backpropagation with sparsity regularization for spiking neural network learning. *Frontiers in Neuroscience*, 16:760298, 4 2022.

[50] Shu Yang, Chengting Yu, Lei Liu, Hanzhi Ma, Aili Wang, and Erping Li. Efficient ann-guided distillation: Aligning rate-based features of spiking neural networks through hybrid block-wise replacement. In *Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition*, 3 2025.

[51] Zhaokun Zhou, Yuesheng Zhu, Chao He, Yaowei Wang, Shuicheng YAN, Yonghong Tian, and Li Yuan. Spikformer: When spiking neural network meets transformer. In *The Eleventh International Conference on Learning Representations*, 2023.

[52] Chenlin Zhou, Liutao Yu, Zhaokun Zhou, Han Zhang, Zhengyu Ma, Huihui Zhou, and Yonghong Tian. Spikingformer: Spike-driven residual learning for transformer-based spiking neural network. *arXiv preprint arXiv:2304.11954*, 2023.

[53] Man Yao, JiaKui Hu, Zhaokun Zhou, Li Yuan, Yonghong Tian, Bo Xu, and Guoqi Li. Spike-driven transformer. In A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, editors, *Advances in Neural Information Processing Systems*, volume 36, pages 64043–64058. Curran Associates, Inc., 2023.

[54] Zhaokun Zhou, Kaiwei Che, Wei Fang, Keyu Tian, Yuesheng Zhu, Senior Member, Shuicheng Yan, Yonghong Tian, and Li Yuan. Spikformer v2: Join the high accuracy club on imagenet with an snn ticket. In *International Conference on Learning Representations*, 1 2024.

[55] Qiaoyi Su, Yuhong Chou, Yifan Hu, Jianing Li, Shijie Mei, Ziyang Zhang, and Guoqi Li. Deep directly-trained spiking neural networks for object detection. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2023.

[56] Guangyao Chen, Peixi Peng, Guoqi Li, and Yonghong Tian. Training full spike neural networks via auxiliary accumulation pathway, 2023.

Acknowledgements

I would like to express my sincere gratitude to my advisor, Professor Eunhyeok Park, for his guidance, support, and insightful feedback throughout my Master's studies. I am also grateful to the committee members, Professor Jungseul Ok and Professor Jae-Joon Kim, for their thoughtful questions and constructive comments that improved the quality of this thesis.

I thank my collaborators on the underlying paper, Byeongho Yu and Jeong Min Oh, for the many fruitful discussions that shaped the ideas presented here. I am grateful to all members of the laboratory for their support and camaraderie.

This work was supported by IITP and NRF grants funded by the Korea government (MSIT) (RS-2019-II191906, RS-2023-00213611, RS-2024-00457882).

Finally, I thank my family for their unwavering support and encouragement.

Curriculum Vitae

Name : Hyunho Kook

Education

2024. 02. – 2026. 08. M.S., Department of Computer Science and Engineering, Pohang University of Science and Technology

2018. 02. – 2024. 02. B.S., Department of Computer Science and Engineering, Pohang University of Science and Technology